Dramatis

"the persons of the drama"

THIRD EDITION

A manual for reading a story by the company it keeps — installing it, using it, correcting it, living with it, and removing it.

Written for authors, editors, translators and literary scholars. No programming knowledge is assumed. Everything you have to type is given in full.

Dramatis 0.1.0 (pre-alpha) · A Kestora Labs product · Apache-2.0 · This manual is CC BY 4.0

Describes the application as built through Phase 6.4, at which point a reading can leave Dramatis for other tools and another tool's reading can come in. Features marked *"not yet"* are planned and named so you know they are coming, not missing by accident.

Contents

What is new in this edition

The second edition described the application through Phase 5, which is where it became correctable. Phase 6 is where it stops being a closed room: a reading can now be written out in formats other tools read, the evidence behind it can be published as standard annotation, a document another tool produced can be read in, and one work can be held in several editions at once. All of it is command-line work — none of the four has a control in the browser window yet, and this edition says so where it matters.

WHAT CHANGED	WHERE
A reading can be exported. GraphML and GEXF for Gephi and its relatives, CSV node and edge lists for a spreadsheet, JSON-LD for a triple store. Each carries the weight basis, the provenance of every claim, and — in whatever room the format has — what the reading is a reading of. Your standing review decisions are applied on the way out, because the export is the copy that gets cited.	Part V, *Exporting a reading*
The evidence is its own export, as W3C Web Annotation. One annotation per quotation, with a `TextQuoteSelector` a tool that has never heard of Dramatis can resolve — and every target says how the quotation is meant to be matched, which is the part a naive consumer would otherwise get wrong.	Part V, *Exporting the evidence*
Someone else's Dramatis document can be read in. A file that validates against the published schema is imported into your project and behaves there like one Dramatis produced. It refuses before it writes anything: every collision is found first, because half an imported reading looks exactly like a reading somebody meant to have.	Part V, *Reading in someone else's document*
One work can be held in several editions. The edition is part of the work's identity, so two editions are two works and neither is a revision of the other. They share one collection and one cast, and a character renamed between editions is *corresponded* rather than merged — merging would caption the 1903 graph with a name the 1903 text does not contain.	Part IV, *Several editions of one work*
A comparison across two editions is attributed to the edition rather than reported as a rewrite, and a renamed character reads as one rename instead of a departure and an arrival. This one is only reachable through the interface the browser talks to; the window still compares within a single work.	Part IV, *Several editions of one work*
Three new commands — `export`, `import` and `correspond` — and one new option, `ingest --edition`.	Part IV, *Every command, at a glance*
A twelfth milestone closes the worked example by publishing the study, which is the step the second edition stopped one short of.	Part VI, *Milestone 12*

PART I

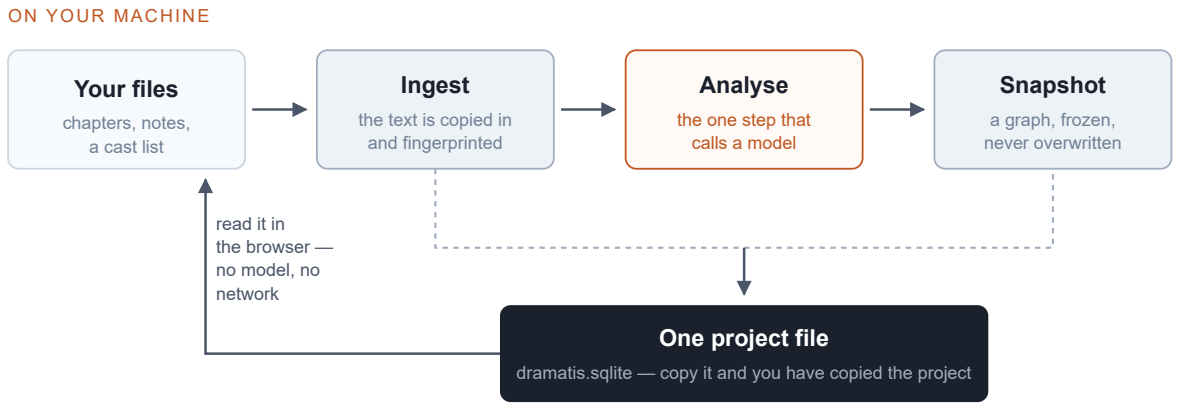
Before you start

Four short chapters. What the thing is, what it promises, the handful of words it uses in a particular way, and the one idea that everything else rests on. Fifteen minutes here will save you an afternoon later.

1 What Dramatis is, and is not

Dramatis reads a narrative work and draws its characters as a network: who is in it, who is close to whom, and what in the text says so. It saves each reading as a dated, unchangeable snapshot, so that when you revise the work you can put the old picture beside the new one and see what moved. And it lets you argue with the result, in a way that sticks.

That is the whole of it. It is a reading instrument, not a writing tool. It never edits your manuscript, never suggests a plot, and never offers an opinion about quality. It answers questions of shape: *who carries this book? which relationships am I actually writing, as against the ones I meant to write? when I cut chapter nine, who fell out of the story with it?*



The shape of the tool. Your files go in; a fingerprinted copy is stored; a model reads that copy once and the result is frozen as a snapshot. Everything after that — opening it, reading it, correcting it, comparing it, exporting it — happens on your machine with no model and no network. Only the orange step ever contacts a model.

What it is good at

- **Seeing a cast whole.** A novel has more people in it than anyone can hold in mind. A picture of them, sized by how much of the book they carry, is a different object from a list of names.

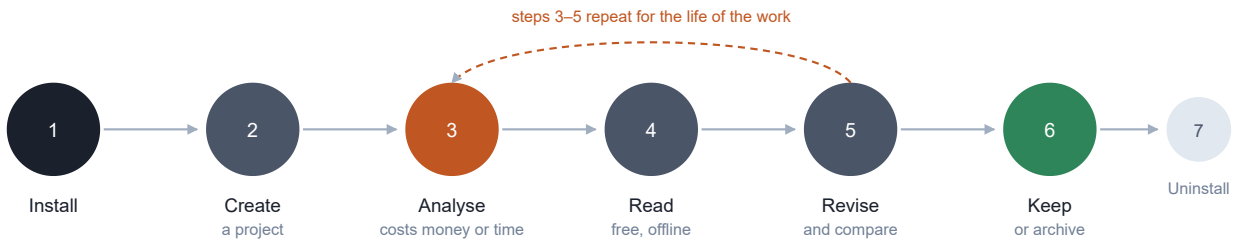
- **Watching a revision land.** Rewrite a chapter, run the analysis again, and Dramatis will tell you which relationships strengthened, which weakened, and which disappeared — and, crucially, whether the change belongs to your rewrite or to the analysis having been done differently.
- **Catching the gap between the plan and the page.** If you keep a cast list or a series bible, Dramatis reads it separately and shows you the relationships you have described but never dramatised, and the ones that took over the book without ever being in the plan.
- **Citable claims.** Every edge in the graph is backed by quotations checked letter-by-letter against your text. You can click any of them and land on the sentence.
- **Being argued with.** A reading is a proposal. You can accept it, reject it, correct it, or declare that two of its characters are one person — and what you decide outlives the reading you decided it about.

What it deliberately is not

- **Not a writing tool.** It reads and reflects. It has no editor and never writes to your manuscript files.
- **Not tied to one author's filing method.** It will not assume that a folder called `draft-2` is a second draft, or that a file called `cast.md` is a cast list. It proposes; you confirm.
- **Not a general knowledge graph.** Characters and their relationships only. Places, objects and events are out of scope.
- **Not a model.** Dramatis ships no artificial intelligence of its own. It orchestrates a model that you supply — either a paid service you hold an account with, or a free one running on your own computer.
- **Not a service.** There are no accounts, no cloud storage, no sharing features and no subscription. It runs on your machine and nowhere else.

The life of a project

This manual follows that life from end to end. Steps 3 to 5 are the loop you will actually live in: write, re-read, correct, compare.



Seven stages, and where they are covered. Installing is Part II; creating, analysing and reading are Part III; arguing with the result is Part IV; publishing it, or taking somebody else's in, is Part V; the loop of revising and comparing is worked through end to end in Part VI; keeping and removing are Part VII.

> A WORD ABOUT THE VERSION
>
> Dramatis is **pre-alpha**. It works, it is thoroughly tested, and the parts described in this manual do what they say. But it is early software: the installation is a developer-style one for now, there is no double-clickable installer yet, and the features named in *What Dramatis cannot do yet* are genuinely absent rather than hidden. Treat a Dramatis project as valuable but not yet as a system of record.

2 The three promises

Three commitments are built into the design rather than into the marketing. They are worth knowing because they explain a lot of behaviour that would otherwise look pedantic.

Your text goes only where you send it

There is no telemetry, no analytics and no phone-home of any kind. Dramatis makes an outbound connection in exactly two situations, both of which you ask for by name: to the model provider you chose, at the moment you ask for an analysis; and to Google Drive, if you have told it that your corpus lives in a Drive folder. Nothing else, ever.

If you choose a model running on your own machine and keep your corpus on disk, nothing leaves the machine at all — you can unplug the network cable and the whole thing still works.

Everything else — opening a project, drawing the graph, following evidence into the source, reviewing, correcting, merging, comparing two drafts, exporting — works with no credential and no network, permanently and by design. This is not a fallback mode; it is the ordinary way the application runs.

Nothing is claimed without a quotation

Every relationship in the graph carries quotations from your text, and every one of them has been checked character-by-character against the source before it was allowed into the record. If the model invents a line, the claim resting on it is *rejected* — not flagged, not shown with a warning, but removed. Where a quotation is genuine but attached to the wrong passage, it is moved to the right one rather than thrown away.

The single concession is whitespace: a quotation spanning a line break in a hard-wrapped file still counts as verbatim. Nothing else is forgiven — not a changed comma, not a straightened quotation mark, not a difference of case.

Nothing is ever overwritten

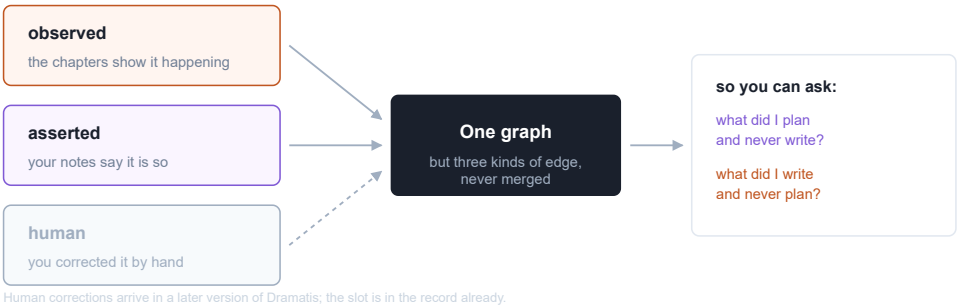
A snapshot is immutable. Analysing the same text again does not replace the earlier reading; it adds a second one beside it. Ingesting an edited chapter does not replace the old chapter; it records a new revision and keeps the old one, because a snapshot taken last month cites text that must still be there to be cited.

This is why your corrections behave the way they do. Correcting a character does not rewrite the snapshot on screen — that snapshot goes on saying what the analysis said, because that is what it said. Your correction is recorded beside it and written into every snapshot built afterwards. Two different facts, kept apart, both visible.

3 Seventeen words you will need

Dramatis uses a small number of ordinary words in a precise way. Learn these and the rest of the manual reads easily. A fuller glossary is at the back.

Project	One file, holding everything: your texts, the cast, every analysis you have ever run, and every decision you have taken about them. Copy the file and you have copied the project entire. It is normally called `dramatis.sqlite` and lives in the folder with your manuscript.
Collection	The circle inside which characters are shared. A project holds one collection. If you put two novels of a series into the same project, a character appearing in both is *one* character, which is what a series wants. An unrelated book belongs in its own project.
Work	A novel, a play, a season — one thing that gets revised over time.
Edition	A published state of a work that supersedes nothing: a first and a revised edition, a translation. Part of the work's identity rather than a point in its history, so two editions are two works, both current and both citable. Most projects never name one.
Source	Where a corpus comes from. A folder on your disk, or a Google Drive folder. A source is contacted only when it is read, never when it is named.
Document	One file. A chapter, a cast list, an appendix. Each has a *role*: it enacts the story, describes it, or is no part of it.
Text revision	The exact state of your manuscript at one moment, fingerprinted. Every time you bring the corpus in again, Dramatis records a new revision and tells you which files changed.
Analysis run	One reading: which model, which instructions, which settings. Two runs with the same configuration are the same reading and can be compared.
Snapshot	A graph, permanently bound to *one* text revision and *one* analysis run. This pairing is the heart of the design; *The two time axes* explains why.
Character	A node. Carries a canonical name and every surface form the text used for them — “Elizabeth”, “Miss Eliza Bennet”, “Lizzy”.
Relation	An edge between two characters, with a weight, a type or two, and its evidence.
Weight and basis	How strong an edge is, *and* what the number counts. A weight without its basis is meaningless, so the two always travel together and Dramatis refuses to compare weights measured on different bases.
Provenance	Where a claim came from: **observed** (the narrative enacts it), **asserted** (your reference material states it), or **human** (you entered or corrected it). Never mixed.
Review status	What a person has made of a claim: **proposed** until somebody looks at it, then **accepted**, **corrected** or **rejected**. Kept beside the snapshot, not in it.
Correction	A statement of what a claim *should* say, which is written into every snapshot built after it. Your correction outranks a later reading, and the disagreement is recorded.
Structure map	Dramatis's proposal about what your corpus holds — which file is story, which is notes, which is no part of the work, which is a later draft of which — with the reason for each guess, waiting for you to confirm or correct it.
Confidence	How sure a reading was of a claim, between 0 and 1. Optional, and absent from everything Dramatis currently produces. An absent confidence is *not* a low one.



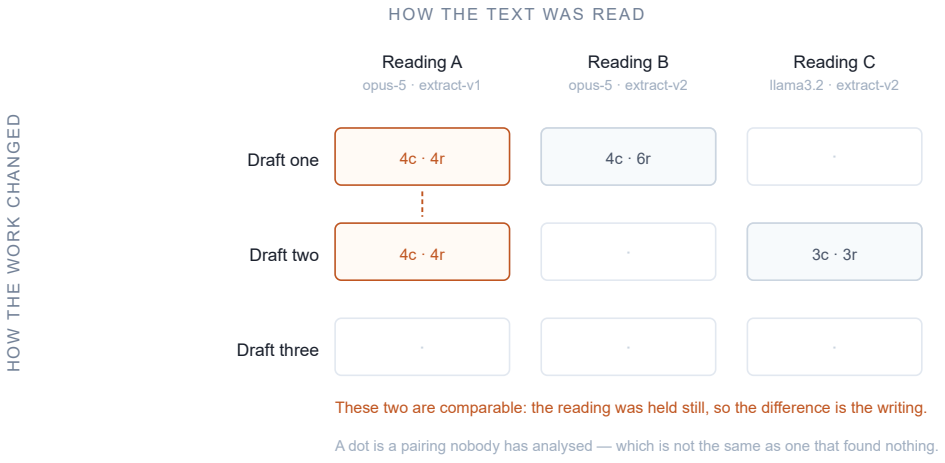
Three sources, one graph, never merged. An edge that your notes declare and an edge that your chapters enact are two different claims about the same pair. Kept apart, they let you ask the two questions on the right. Merged into one number they would answer neither. Anything you correct by hand becomes the third kind, and leaves that comparison rather than pretending to be evidence about the corpus.

4 The two time axes

This is the one idea in Dramatis worth reading twice. Everything about the way snapshots are stored and compared follows from it.

A character graph can change for two entirely different reasons. Either **the work changed** — you rewrote a chapter, cut a subplot, gave a minor character three more scenes. Or **the reading changed** — you ran the analysis with a better model, or at a higher effort setting, or with revised instructions.

If a tool cannot tell you which of those happened, its comparisons are worthless. You would have no way of knowing whether your rewrite worked or whether you simply paid for a smarter reader. So Dramatis never collapses the two. A snapshot is bound to a text revision *and* an analysis run, both always recorded, and the browser lays your snapshots out on a grid with one axis for each.



Reading the grid. Down a column, the reading is held still and the text varies: any difference is your writing. Across a row, the text is held still and the reading varies: any difference is the analysis. A snapshot in a different row *and* a different column can be blamed on neither, and Dramatis will say so in plain words rather than pretend.

In practice this means one habit is worth forming: when you re-analyse after a rewrite, tell Dramatis to hold the reading exactly as it was last time. There is a single option for it, `--like`, and *Comparing two*

drafts shows it in use.

PART II

Installing

Dramatis is pre-alpha, so installation is still a developer-style one: a few commands in a terminal window rather than a double-clickable installer. A signed desktop application is planned. What follows assumes you have never used a terminal and explains every step.

5 What you need first

A terminal

A terminal is a window where you type commands instead of clicking. You already have one.

- **macOS** — press `⌘ Space`, type *Terminal*, press Return.
- **Windows** — press the Start key, type *PowerShell*, press Return.
- **Linux** — you know where it is.

Throughout this manual, anything shown in a dark box is typed into that window and followed by Return. A pale box is what the application printed back; do not type those. Lines beginning with `#` are comments for you, not commands.

Python 3.11 or later

Dramatis is written in Python. Check whether you already have a suitable version:

```
python3 --version
```

If that prints `Python 3.11.x` or higher, you are set. If it prints something older, or an error, install a current Python from python.org/downloads — take the default options, and on Windows tick *"Add Python to PATH"* on the first screen of the installer. On Windows the command is usually `python` rather than `python3`.

Node.js 20 or later

The browser view is built from source at installation time, which needs Node.js. Check with:

```
node --version
```

If it is missing, install the LTS release from nodejs.org. You will never need to think about Node again after installation.

Git

Used once, to fetch the source. `git --version` will tell you whether you have it; if not, git-scm.com/downloads.

Somewhere for the model to come from

Analysis needs a language model. You have two choices and they have quite different consequences; *Choosing where the thinking happens* lays them out. You do not need to decide before installing.

6 Installing, step by step

> THE NAME ON THE PACKAGE INDEX IS NOT `DRAMATIS`
>
> An unrelated actor library has held `dramatis` on the Python package index since 2008, so `pip install dramatis` fetches a stranger's package. Dramatis publishes as **`dramatis-personae`**. The command you type and the folder you install from are both still called `dramatis`; only the published name differs. The instructions below install from the source checkout and sidestep the question entirely.

1 · Fetch the source

Choose where it should live and clone it there:

```
# macOS and Linux
cd ~
git clone https://github.com/kestora-labs/dramatis.git
cd dramatis
```

```
# Windows PowerShell
cd $HOME
git clone https://github.com/kestora-labs/dramatis.git
cd dramatis
```

2 · Make a private workspace for it

A *virtual environment* is a folder holding this application's dependencies so they cannot disturb anything else on your computer. Create and activate one:

```
# macOS and Linux
python3 -m venv .venv
source .venv/bin/activate
```

```
# Windows PowerShell
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Your prompt will now begin with `(.venv)`. That is how you know the workspace is active. It stays active until you close the window; whenever you come back to Dramatis in a new terminal, run the `activate` line again first.

> IF WINDOWS REFUSES TO RUN THE ACTIVATE SCRIPT
>
> PowerShell blocks scripts by default. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer *Yes*, and try again.

3 · Install the application

The square brackets ask for two optional extras: `anthropic`, the adapter for the hosted model, and `serve`, the browser view. Include both; they are small and you will want them.

```
pip install -e ".[anthropic,serve]"
```

Reading a Google Drive corpus needs nothing extra — it is done with what Python already ships, deliberately, so that the claim about where bytes go stays cheap for anyone to check.

4 · Build the browser view

Two commands, once, taking a minute or so:

```
npm ci --prefix web
npm --prefix web run build
```

This turns the source of the browser interface into files the Dramatis server can hand to your browser. If you skip it, every command still works but `dramatis serve` will show a page telling you the client has not been built. Run these two again whenever you update the source.

5 · Confirm

```
dramatis --version
```

Should print something like `dramatis 0.1.0.dev0`. If it prints *command not found*, the virtual environment is not active — go back to step 2.

7 Choosing where the thinking happens

Analysis is the one step that needs a language model, and you decide which. The choice is not permanent — it is made per analysis — but it has real consequences for cost, quality and

privacy, so make it deliberately.

	A HOSTED MODEL (ANTHROPIC)	A MODEL ON YOUR MACHINE (OLLAMA)
Your text	Is sent to the provider, in passages, as the analysis runs.	Never leaves the computer. Works with the network cable out.
Cost	Paid per use, on your own account with the provider.	Free. Costs electricity and time.
Quality	Currently much better, especially at recognising that two names are one person.	Usable; depends entirely on the model you download.
Speed	Minutes for a novel.	Minutes to hours, depending on your machine.
Set-up	An account and an API key.	Install one application, download one model.

Option A — a hosted model

Create an account at console.anthropic.com and generate an API key. Then make it available to Dramatis by putting it in the environment before you run an analysis:

```
# macOS and Linux
export ANTHROPIC_API_KEY=sk-ant-...
```

```
# Windows PowerShell
$env:ANTHROPIC_API_KEY = "sk-ant-..."
```

That setting lasts until you close the window. To make it permanent, add the same line to your shell profile — on macOS that is usually `~/.zshrc`.

> **AN API KEY IS A PASSWORD**
>
> Anyone holding it can spend your money. Do not paste it into a document, an email, or a file you will later commit to version control.

Option B — a model on your own machine

Install Ollama — it is an ordinary application with a normal installer for macOS, Windows and Linux. Then download a model:

```
ollama pull llama3.1
```

That is the whole set-up. There is no key, no account, and no charge. From then on, add `--provider ollama` to any analysis and the manuscript stays on your machine.

> **A LOCAL MODEL IS ONLY LOCAL IF IT IS LOCAL**
>
> Ollama can be pointed at another computer. If yours is, Dramatis notices and prints a warning saying your text is about to leave the machine, so you find out before the analysis rather than after it.

Two things worth knowing about local models

They used to read less than they were sent. Ollama does not complain when a prompt is longer than the context window it has been given: it silently discards the *beginning* of the prompt and answers from what is left. Measured against a real local model, 83% of a passage went missing and the reply came back looking perfectly normal — a confident answer to a question nobody asked. Dramatis now works out and asks for a window big enough for each call, so this cannot happen. If you have an older checkout, update it.

They can be slower than the ten-minute limit allows. Each call to a local model is given ten minutes, and a large passage on a machine without a fast graphics card can exceed that. The error message suggests raising the timeout, and at present there is no option that does. The remedies that work are a smaller or faster model, or a machine with more to give.

Which to pick

If you are analysing a published or public-domain text, use the hosted model: it is better and the privacy question does not arise. If you are analysing an unpublished manuscript that you are not comfortable sending anywhere, use Ollama and accept a rougher reading. Many people do both — a local pass while drafting, a hosted pass when it matters.

8 Checking that it worked

Dramatis ships with a hand-authored example graph. Validating it exercises the schema and its checks without needing a model, a key, or a network:

```
dramatis validate fixtures/a/snapshot.json
```

```
ok    fixtures/a/snapshot.json
```

And the same command on a deliberately broken example:

```
dramatis validate fixtures/a/invalid/self-relation.json
```

```
FAIL  fixtures/a/invalid/self-relation.json — 1 problem
      [reference] /relations/0: a relation may not join a character to itself
```

If both behave that way, your installation is sound.

9 Installing with Docker instead

If you already use Docker, there is an image that bundles the application, the browser view and everything it needs. It is an alternative to the rest of this part, not an addition.

```
docker build -t dramatis .
docker run --rm -p 127.0.0.1:7373:7373 -v "$PWD/project:/data" dramatis
```

The project file lands at `./project/dramatis.sqlite` on your machine, and the interface is at `http://127.0.0.1:7373`.

> **WHY THE ADDRESS IS WRITTEN OUT IN FULL**
>
> Inside a container, "this machine only" is unreachable from outside it, so the server inside binds to everything. Publishing the port to `127.0.0.1:7373` rather than `7373` is what keeps your manuscript off the office network. Write the address out; the short form is not the same.

PART III

Using it

Making a project, telling Dramatis what your files are, running a reading, and then the several ways of looking at the result. Screenshots are from real analyses of *Pride and Prejudice*.

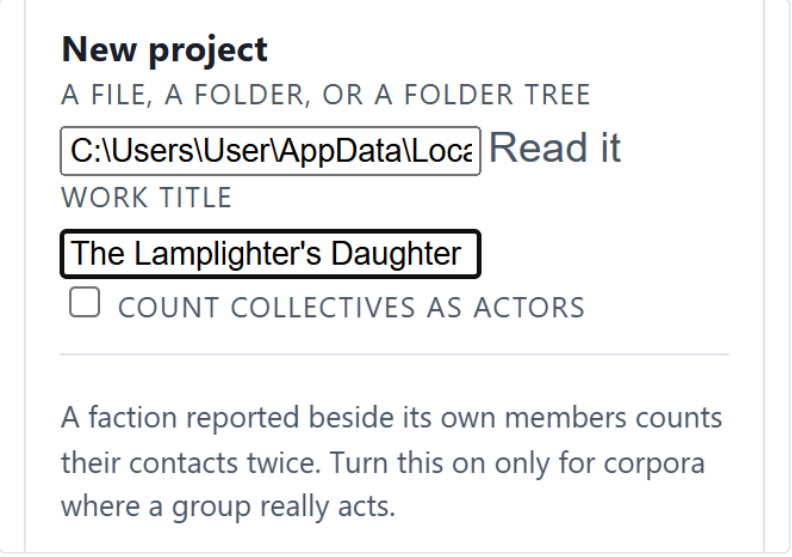
10 Making a project in the browser

The friendlier of the two routes. You point the server at a project file that does not exist yet, open it in your browser, and build the project by clicking.

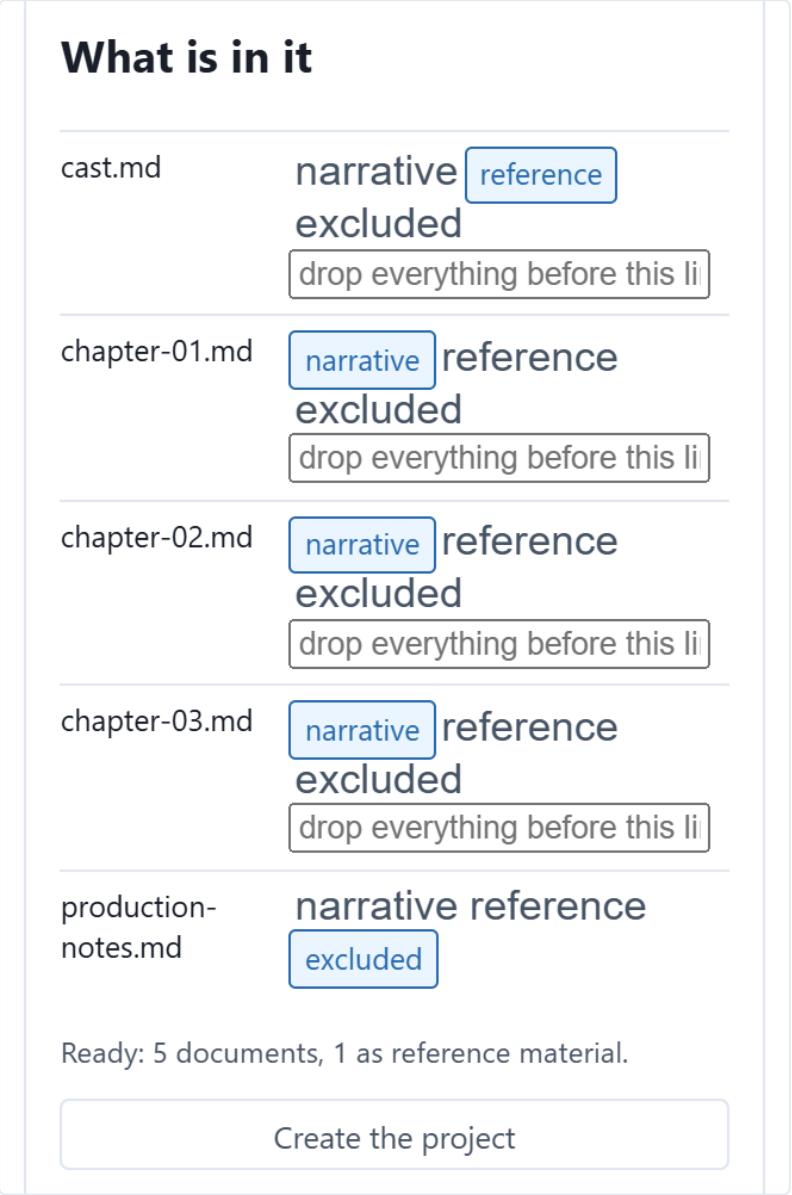
Put your manuscript files in a folder, then, from inside that folder:

```
cd ~/writing/my-novel
dramatis serve --store dramatis.sqlite
```

Nothing is created yet. Dramatis says as much, prints an address, and waits. Open `http://127.0.0.1:7373` in your browser. Because the project holds nothing, the **New project** panel opens itself.



Where the work is, and what to call it. A path, a title, and one question about the whole study. Pressing *Read it* reads the folder; it calls no model and writes nothing.



The screen that matters. Dramatis lists what it found and asks one question about each file: does this *enact* the story, *describe* it, or is it no part of the work at all? Nothing is assumed from the filename — you are asked, and asked once. It will not let you create the project until every file has an answer.

The panel has four things in it:

- **The source.** A single file, a folder, or a folder of folders. Type the full path.
- **The work title.** Optional; taken from the filename if you leave it blank.
- **Count collectives as actors.** Whether a group — a family, a crew, a faction — is a character in its own right, or only the people named in it. Leave it off for a novel. Turn it on only where a group genuinely acts as one, as in a serial about rival houses. It applies to the whole project, and snapshots taken either side of a change to it answer different questions and cannot be compared.
- **A role for each document**, plus an optional line to drop everything before. The next chapter is about these.

Create the project writes the file, records your settings, saves your answers about the documents, and reads the text in. It calls no model and costs nothing. Analysing is a separate act, and deliberately so — you can build a project, look at what it holds, and abandon it without having spent anything.

11 Making a project from the command line

The same thing in one line. From inside the folder holding your manuscript:

```
dramatis ingest . --work "My Novel" --creator "Your Name"
```

Or for a single file:

```
dramatis ingest draft.txt --work "My Novel"
```

Dramatis reports what it did:

```
ingested: rev:568a07982816 (4 documents, 3,784 characters, sha256 568a07982816...)
  4 added
  store       dramatis.sqlite  (new project)
  collection  col:my-novel
  work        work:my-novel
  revision    rev:568a07982816

  added     cast.md
  added     chapter-01.md
  added     chapter-02.md
  added     chapter-03.md
```

Four things about `ingest` are worth knowing:

- **It is the only command that creates a project.** Every other command refuses to run rather than quietly starting an empty project you did not ask for — which is what saves you when you run a command from the wrong folder.
- **Running it twice is harmless.** The revision identifier comes from the content of the files, so ingesting unchanged text again changes nothing.
- **It tells you what moved.** On a second ingest, each file is reported as *added*, *changed* or *unchanged*, which is what later lets a comparison say that the graph moved because chapter three was rewritten.
- **Always pass `--work`.** If you ingest `.` without one, the title is derived from a folder name that a bare dot does not supply, and you get a work called *untitled*.

> WHERE THE PROJECT FILE IS LOOKED FOR
>
> Every command except `ingest` searches for `dramatis.sqlite` in the current folder and every folder above it, the way version control finds its own directory. So commands work anywhere inside a project, and `dramatis status` will always tell you which file it found and how.

One project, one collection

A second work ingested into the same project joins the same collection and shares its cast — which is exactly what a series or a shared universe wants:

```
dramatis ingest book-one.txt --work "Meteor Girl" --collection "Golden Age"
dramatis ingest book-two.txt --work "The Spark"        # joins the same collection
```

An unrelated book belongs in its own project file. Dramatis refuses to merge two casts into one namespace, because that is not something you could undo.

12 Saying what each file is

Before it reads anything, Dramatis wants to know what it is looking at. It proposes; you confirm. It never infers a filing convention, because filing conventions are personal and a wrong guess would silently corrupt the reading.

Looking at a corpus costs nothing

```
dramatis structure .
```

This reads no model and touches no network. It prints one block per file, each proposal followed by the evidence for it:

```
/writing/lamplighter - 4 documents

  cast.md  (1,022 characters)
    role         unknown
                 needs the text read, not the folder named
    addressing   section
                 blank-line sections
    revision of  (none)
                 no earlier document is named cast.md

  chapter-03.md  (910 characters)
    role         unknown
    ...
```

The three questions

- **Role — narrative, reference, or neither?** A *narrative* document enacts the story: chapters, scenes, transcripts. A *reference* document describes it: a cast list, a series bible, an outline, a wiki export. The distinction is not cosmetic — relationships found in the two are kept apart all the way through, and the disagreement between them is one of the most useful things Dramatis will show you. The third answer, *excluded*, is below.
- **Addressing — how is a position in this file named?** At present Dramatis divides text on blank lines and calls each block a *section*. Richer structures come later.
- **Revision of — is this a later state of another file?** Proposed from the filename and how much text two files have in common, and shown with a percentage so you can judge it.

Files that are no part of the work

Real manuscript folders hold things that are neither story nor notes about the story: a production schedule, a to-do list, a style guide, a sheet of cover briefs, a specification for a file format. Forcing them to be "reference material" would put their vocabulary into your cast.

Mark them **excluded** and they are left out of the revision entirely — not stored, not hashed into the fingerprint, never sent anywhere. In the browser it is a third button beside the other two. On the command line:

```
dramatis structure . --set production-notes.md=excluded --confirm
```

Answering

In the browser, you answer by clicking the roles in the New project panel. From the command line, you correct and confirm in one go:

```
dramatis structure . --set cast.md=reference --confirm
```

`--confirm` saves your answers against that corpus, so you are never asked again; later ingests apply them automatically. It refuses while any document still has an unknown role, because half a map is worse than none. If your mind changes:

```
dramatis structure . --forget
```

Letting a model read the corpus for you

For a large or unfamiliar corpus, a model can read the documents and propose roles itself. This is the only part of `structure` that costs anything, and you have to ask for it:

```
dramatis structure . --ask
```

It still writes nothing. You look at the proposals, correct what is wrong, and confirm separately.

Leaving out a preface

Editions of classic texts often bind a critical introduction into the same file as the novel. Its characters are a modern scholar and their colleagues, and they have no business in your graph — and analysing them costs money.

In the New project panel, each document has a field reading *"drop everything before this line"*. Paste the first line of the actual narrative into it. Everything above is dropped before the text is stored, so it never reaches the model at all.

13 When the corpus lives in Google Drive

Many working corpora are Google Docs in a Drive folder. Until recently the only way to analyse one was to export the lot by hand — and to do it again on every revision, which quietly defeats the point of revisions. Dramatis can now read the folder where it is.

What this does and does not do

Dramatis asks Google for **read-only** access, and nothing else. It never writes to your Drive, never creates anything there, and never shares anything. Google enforces that, rather than Dramatis promising it: the consent screen asks for the read-only scope and no other, so the grant you give cannot be used for more.

Reading a Drive folder does mean your text travels: from Google to your machine, over an encrypted connection, when you ingest. That is the second and last of the two situations in which Dramatis makes an outbound connection, and like the first you ask for it by name — a local path that happens to look like a Drive address is read as a path, never as a Drive folder.

Consenting, once

Dramatis ships no Google credentials of its own, and this is deliberate: an application identifier published in an open-source repository is a shared secret with the entire internet. So you create one, once, and it is yours.

1. Go to the Google Cloud console and create a project — the name is for you alone.
2. Enable the **Google Drive API** for it.
3. Create an **OAuth client ID** of type **Desktop app**, and download the `client_secret_….json` file it offers.

Then, once, on each machine you want to use:

```
dramatis authorise --client-secret ~/Downloads/client_secret_....json
```

Your browser opens, Google asks whether you consent to read-only access to your Drive, and the answer comes back to a listener on your own machine. Nothing is typed into a web page belonging to anyone else.

To check where you stand, or to withdraw:

```
dramatis authorise --status
dramatis authorise --forget
```

> WHERE THE CREDENTIAL IS KEPT, AND WHY IT MATTERS
>
> The refresh token is written into your own configuration directory, with owner-only permission where the platform has any — and **never** into the project file. That is the whole point: a project file is a thing people send to each other, and a credential must not travel in one. `--forget` deletes the local copy; it does not revoke the grant at Google, which you do in your Google account settings.

Reading the folder

Copy the folder's address out of your browser and hand it over:

```
dramatis structure --drive https://drive.google.com/drive/folders/<id>
dramatis ingest --drive https://drive.google.com/drive/folders/<id> --work "My Novel"
```

Everything downstream is identical to a folder on disk:

- **Google Docs are exported as Markdown**, so their headings survive and can be read as structure. A file you uploaded is downloaded as it stands and judged by its suffix — the same rule a local folder uses.
- **Anything unreadable is named, with its reason.** A spreadsheet, an image, a PDF: each is listed as skipped and why. A revision quietly missing a chapter is a graph missing a character with nothing on screen to say so, which is the one outcome a source may never produce.
- **Re-ingest to get a second revision.** Edit the Docs, run the same command again, and each document is reported as added, changed or unchanged, exactly as a local folder is.
- **Renaming the folder in Drive changes nothing.** A corpus is identified by where it is, not by what it is called, so a rename does not produce a phantom second corpus.
- **A confirmed structure map is remembered against the Drive folder**, so a later ingest reuses your answers rather than asking again.

> TWO ROUGH EDGES, HONESTLY
>
> Where several documents in a folder share a name, only the first is read and the others are reported as colliding — and at present that message names the wrong one of the pair, so it tells you less than it should. And where a file is skipped for one reason, later files of the same name can be reported with a different and untrue reason. Both are recorded as known defects. If a Drive ingest reports collisions, check the document count against what you expect before analysing.

14 Running an analysis

The one step that calls a model, costs money or time, and produces a snapshot. Everything else in Dramatis is free.

You need the identifier of the text revision to read. `dramatis status` lists them:

```
dramatis status
```

```
project     /writing/lamplighter/dramatis.sqlite
            found by searching upward from /writing/lamplighter

settings    collectives_are_actors = false

collection  The Lamplighter's Daughter  (col:the-lamplighter-s-daughter)
registry    4 character(s)

work        The Lamplighter's Daughter  (work:the-lamplighter-s-daughter)
            2 revision(s), 2 snapshot(s)
  snap:bb9660a6fe4b  2026-08-19T06:52:39+00:00  4 characters, 4 relations  draft one
  snap:243d20a2f047  2026-08-19T07:14:38+00:00  4 characters, 3 relations  draft two
```

Then analyse it:

```
dramatis analyse rev:568a07982816 --label "draft one"
```

Or, with a model on your own machine:

```
dramatis analyse rev:568a07982816 --provider ollama --label "draft one"
```

What comes back:

```
snapshot    snap:bb9660a6fe4b
  revision  rev:568a07982816
  run       run:d5f0082cf9e2
  model     llama3.2:3b
  characters 4
  relations  4
    observed 3  (interaction_passages)
    asserted 1  (asserted_statements)
  rejected   6 of 37 quotations as not verbatim
```

Read that last block carefully; it is the honesty of the tool on display.

- **Observed and asserted are counted separately**, with the basis of each. Three relationships were shown happening in the chapters; one was stated in the cast list. A single total would hide the distinction, and the distinction is the finding.
- **Six quotations were rejected.** The model produced them; they were not found in the text; they were removed along with the claims resting on them. If more than a quarter of them fail, the whole run is refused rather than delivering a plausible but thin graph.

The options that matter

`--provider ollama`	Use a model on this machine. No key, no network, nothing leaves.
`--model`	Name a specific model rather than the provider's default.
`--effort`	How hard the model should work per passage: `low`, `medium` (the default), `high`, `xhigh`, `max`. Higher costs more and reads better. Ollama has no such dial, and Dramatis says so rather than pretending.
`--like SNAPSHOT`	Copy the settings recorded by an earlier snapshot, so a re-run after a rewrite holds the reading still. **The most important option here.**
`--label`	A human name for the snapshot, such as “draft two, after the cut”.
`--checkpoint FILE`	Record every model call to a file, so an interrupted analysis of a long novel resumes instead of paying for the same calls twice.

> **A CHECKPOINT FILE HOLDS YOUR TEXT**
>
> It contains the full passages sent to the model. Keep it beside the project, and never in version control or a shared folder.

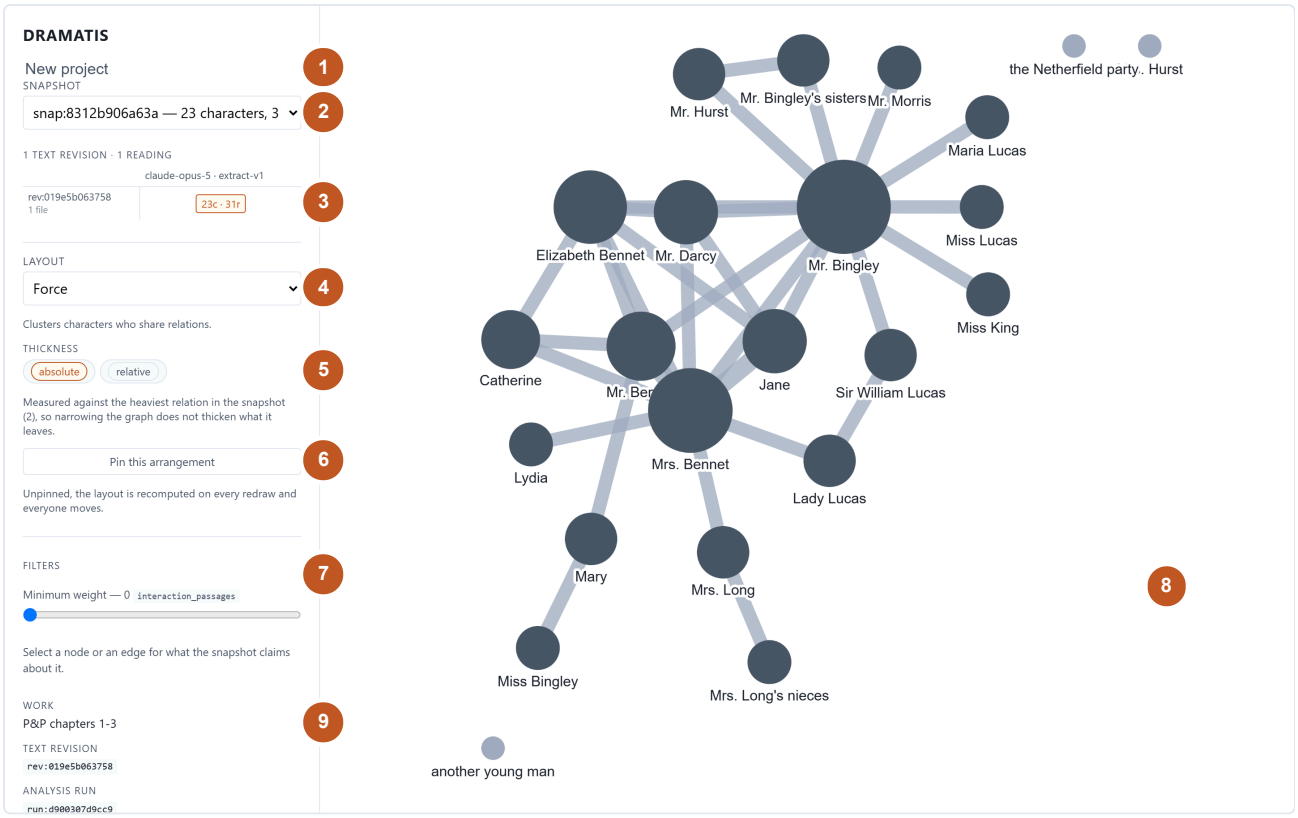
An analysis also applies everything you have decided since the last one: corrections you made, characters you merged or split. Part IV is about those.

15 The window, part by part

From inside your project folder:

```
dramatis serve
```

Then open `http://127.0.0.1:7373`. The server binds only your machine's own loopback address, so nothing on your network can reach it. Stop it with `Ctrl-C`.



The whole interface. One sidebar of controls and one stage. There are no menus, no toolbars and no hidden panels — everything the application can do in a browser is on this screen. Shown: the first three chapters of *Pride and Prejudice*, 23 characters and 31 relationships.

1. **New project** — build a project from files, without the terminal.
2. **Snapshot chooser** — every reading the project holds. Switching is instant and free.
3. **The lineage grid** — your snapshots on the two time axes. Click to open; shift-click a second to compare.
4. **Layout** — how the characters are arranged. Five choices, each with a note saying what it is good for.
5. **Thickness** — whether edge width is measured against the whole snapshot or only what is on screen.
6. **Pin** — keep this arrangement so the graph reopens looking the same.
7. **Filters** — narrow by weight, relationship type, or provenance. Only the controls this snapshot can actually use appear.
8. **The graph** — click a circle for a character, a line for a relationship, the background to deselect. Drag to move, scroll to zoom.
9. **Provenance of the picture** — which work, which text revision, which analysis run, which model, which version of the instructions, and what confidence the reading recorded. This is what makes the graph citable.

16 Reading the graph

What size and thickness mean

A circle is a character, sized by how much of the work they carry. A line is a relationship, thickened by its weight. Both are on a square-root scale, and that is a deliberate distortion: interaction counts in fiction are wildly skewed, and on a straight scale the two leads would be drawn as ropes and everyone else as indistinguishable hairs.

A pale circle is a character with no relationships left in view — either they never had any, or your filters have removed them all.

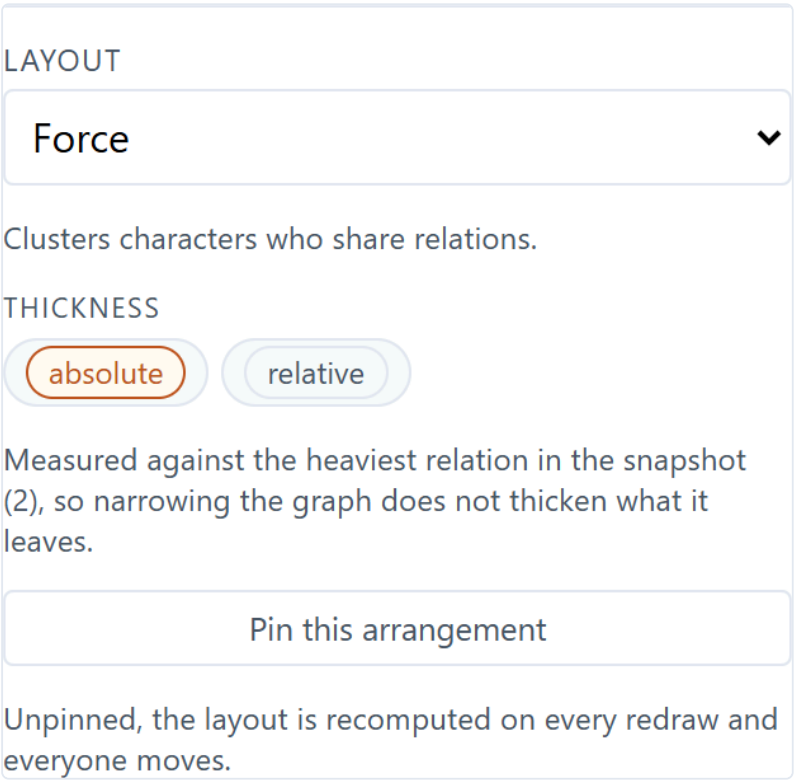
Absolute and relative thickness

Absolute measures every edge against the heaviest relationship in the whole snapshot, so narrowing the graph does not fatten what remains. **Relative** measures against the heaviest edge currently on screen, so a filtered view uses its full range. Use absolute when comparing; relative when studying a corner.

Where a snapshot holds relationships measured on different bases — some counted from passages, some from statements in your notes — Dramatis measures within each basis separately and says so beneath the control. It will not put two incomparable numbers on one scale.

Layouts

Five arrangements: **Force** (the default, clusters characters who share relationships), **Circle**, **Concentric**, **Hierarchy** and **Grid**. Each has a one-line note under the chooser saying what it shows well.



Layout, thickness, and the pin. The note under each control tells you what the setting actually does to the picture, so you do not have to experiment to find out.

Pinning

A force layout has no memory: every redraw scatters the arrangement you have just finished learning. *Pin this arrangement* keeps it. The graph reopens exactly as you left it, and if you drag a character somewhere better, the new position is kept too.

A pin belongs to one snapshot and lives in your browser, not in the project file. Two consequences: a colleague opening your project sees their own arrangement, and clearing your browser's storage loses the pin but nothing else.

Confidence, where a reading recorded it

A reading may say how sure it was of each claim, as a number between 0 and 1. Where it did, Dramatis draws anything below 0.50 with a ⬚ dotted line, and states the threshold on screen so that a reader who disagrees with it knows it was applied.

Two cautions, both important. **An absent confidence is not a low one.** An edge the reading said nothing about is drawn exactly as a confident one is, because "I did not say" and "I was unsure" are different statements and only one of them was made. And **nothing Dramatis currently produces records confidence at all** — the instructions never ask a model how sure it was. The sidebar will tell you so, in as many words. The feature is there for readings that come from elsewhere, and for the day the instructions start asking.

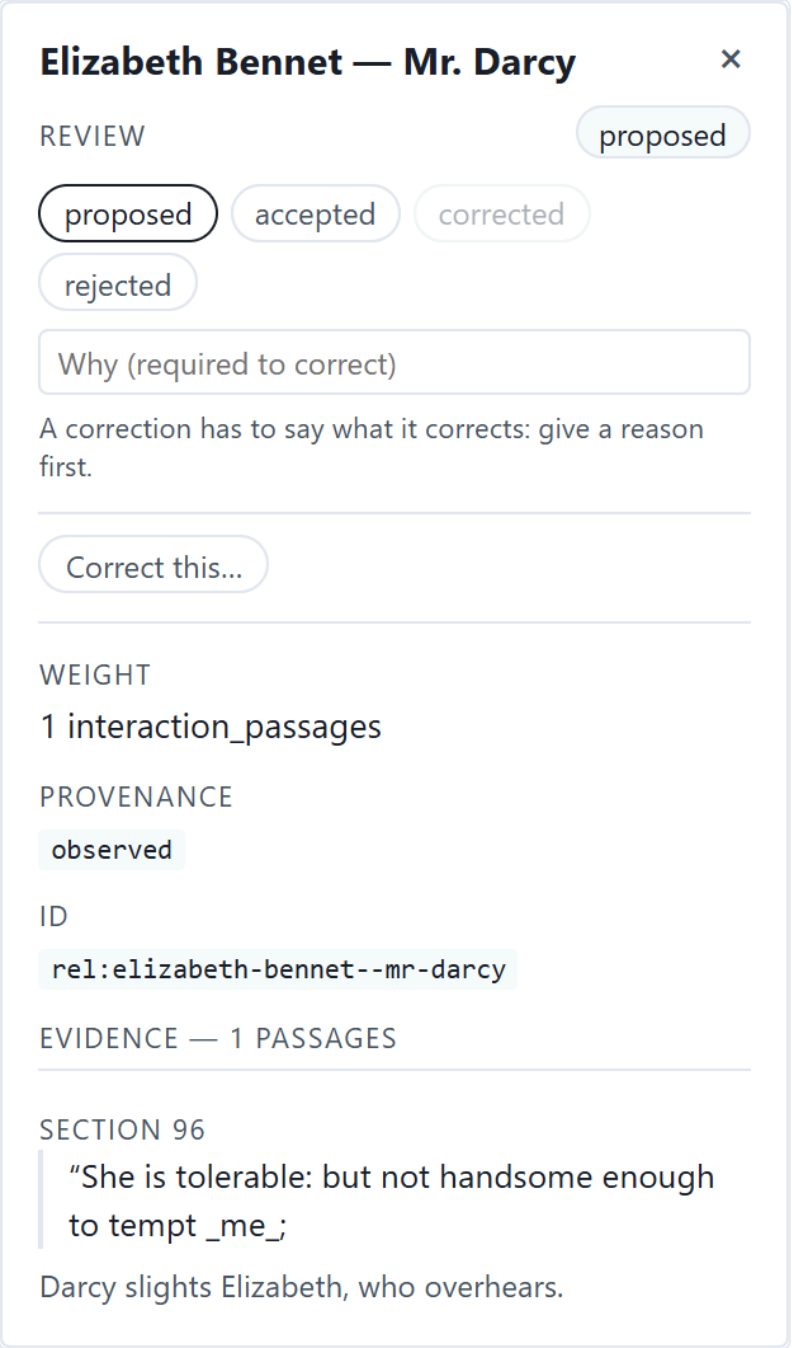
17 Asking a character a question

Click a circle. The sidebar fills with what the snapshot claims about that person.

Mr Darcy, from a full-novel reading. The *also known as* list is the work of alias resolution: every surface form the novel used, gathered onto one person. “my master”, “our visitor” and “I (the letter writer)” are all Darcy, and catching that is most of what separates a useful graph from a list of near-duplicate names.

- **Also known as** — every name the text used for them.
- **Kind** — a person, or a collective.
- **Relations** — how many other characters they are connected to.
- **Provenance** — observed, asserted, or human.
- **Review** — what you have made of the claim, and the buttons to change it. Part IV is about this.
- **ID** — the stable identifier, the same across every snapshot in the project.

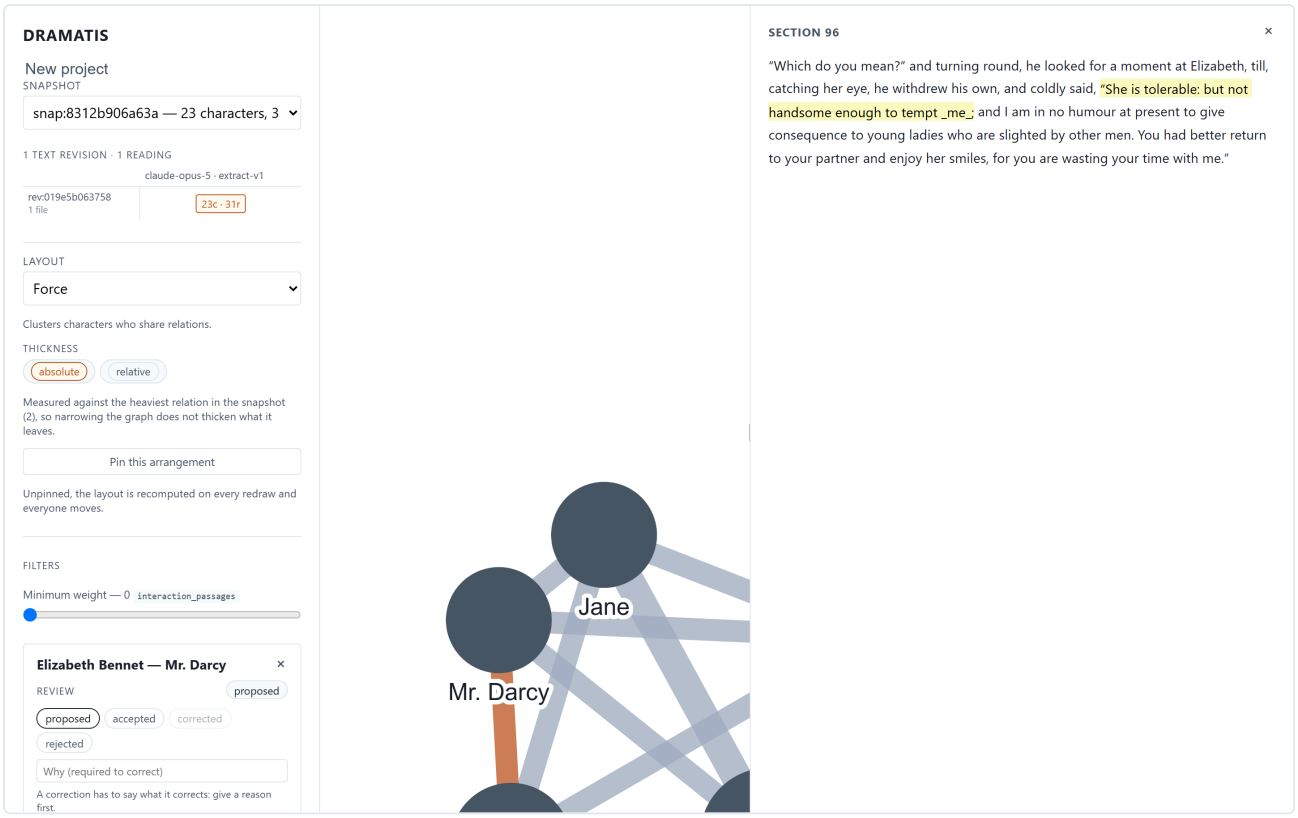
Click a line instead and you get the relationship, with its evidence.



Elizabeth Bennet and Mr Darcy, three chapters in. One passage of contact so far, and the panel names it, quotes it, and says why it counts. Every claim in Dramatis looks like this underneath. The review buttons sit above the claim, because “is this right?” is asked of the whole thing rather than of its last row.

18 Following the evidence into the text

Click any quotation in the evidence list and your source opens beside the graph at that passage, with the quoted words highlighted and scrolled into view.



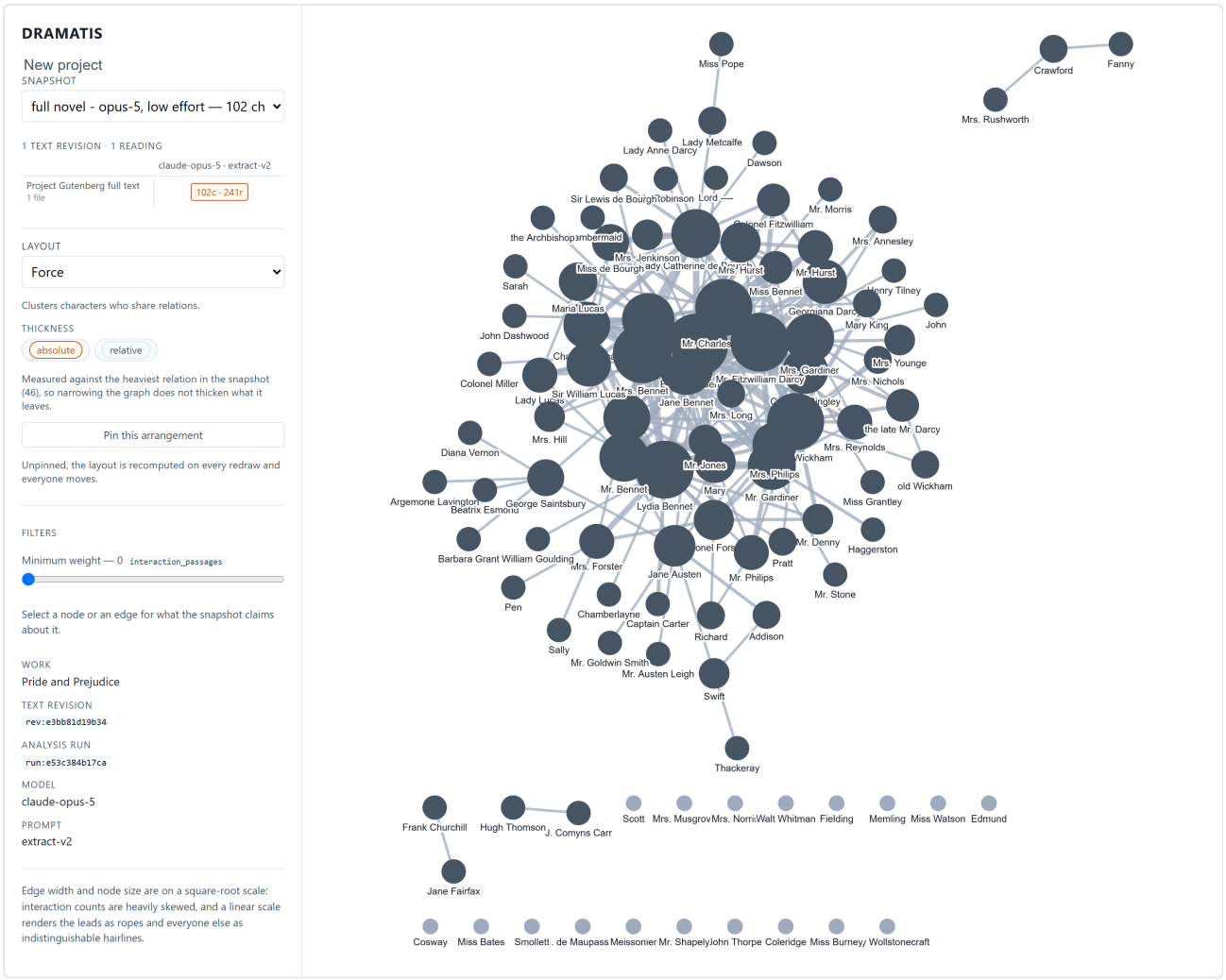
From an edge to a sentence in two clicks. The highlight is computed by the server against the stored text, not searched for by the browser — which is why it works on quotations that span the line breaks of a hard-wrapped file.

The reader is candid about how well it found the passage, and this matters when you have edited the text since the analysis was run:

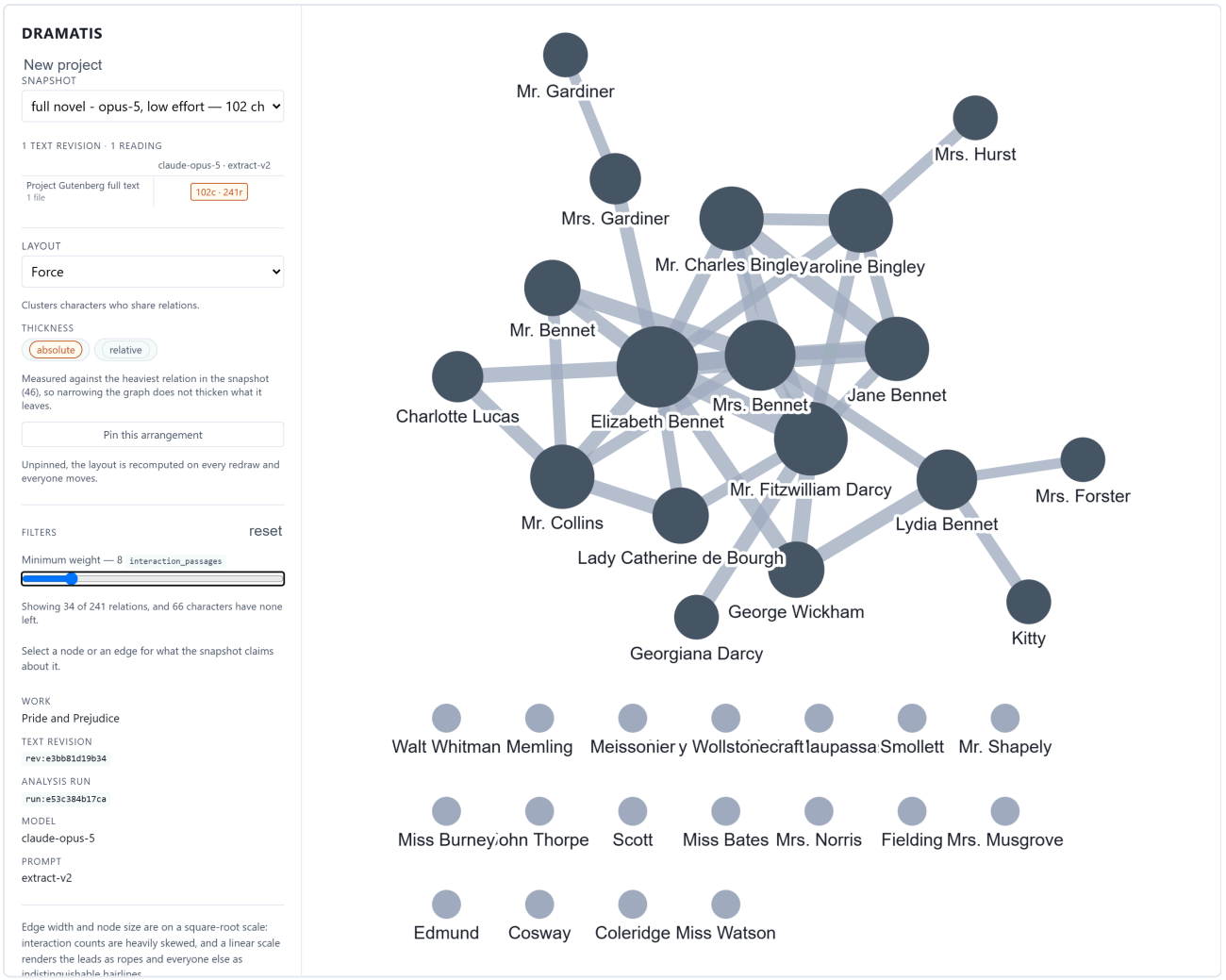
- **Found exactly.** No remark. The quotation is in the current text, word for word.
- **Found approximately.** The highlight is drawn in a different style and a note says the match was fuzzy — you edited inside the quoted sentence, and this is the closest passage. Weigh it accordingly.
- **Moved.** A note says where the evidence used to point. Inserting three paragraphs earlier in a chapter shifts everything after it, and Dramatis simply finds the sentence again.
- **Not found at all.** The passage is shown without a highlight and a plain error says the quotation is no longer in this revision. You rewrote it past recognition. Saying so is the useful answer; silently pointing at whatever scored highest would make every citation in the project unfalsifiable.

19 Narrowing a crowded graph

A whole novel is a hairball. That is not a defect of the drawing — it is what a novel's cast looks like — but it is not readable, and the filters exist to cut through it.



All of *Pride and Prejudice*: 102 characters, 241 relationships. Every walk-on part, every servant, every author named in the Project Gutenberg apparatus at the end of the file. True, and unreadable.



The same snapshot with the weight slider at 8. Thirty-four relationships of 241 remain and the novel appears: the Bennets, the Bingleys, Darcy, Wickham, Collins, Lady Catherine. The tally beneath the slider states exactly what is being hidden — 66 characters now have no relationships left in view.

Three filters, and each appears only if this snapshot has something for it to do:

- **Minimum weight.** A slider. The label names the basis being counted, so you always know what the number means. The graph is rebuilt when you let go, not while you drag.
- **Relationship type.** Chips for whatever vocabulary this analysis produced — kinship, courtship, rivalry, employment. The vocabulary is open, so it varies by work, and no snapshot is forced into a closed list.
- **Provenance.** Observed, asserted, human. Show only what your notes declare, only what the chapters enact, or only what you have corrected by hand.

Filters are per snapshot and reset when you switch, because "kinship" carried across to a snapshot that never heard the word would silently empty the graph. There is a *reset* control whenever anything is narrowed.

20 Comparing two drafts

The reason the application exists. One snapshot is a picture of a cast; two snapshots are a story about how the cast moved.

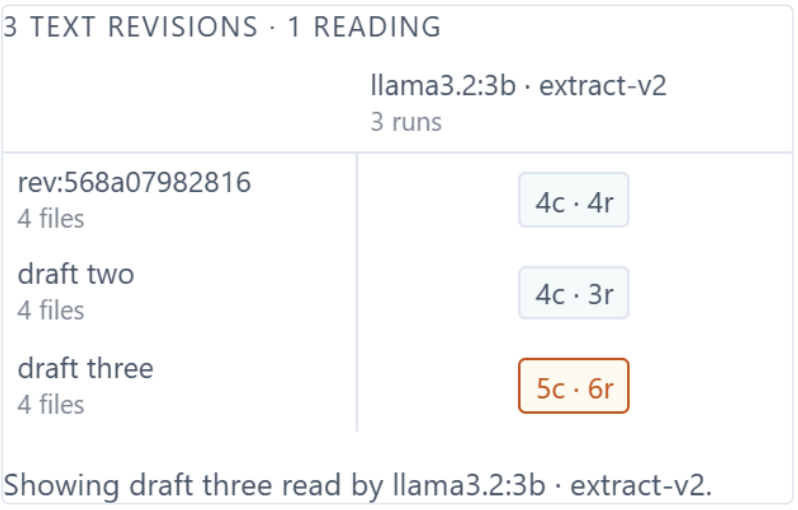
Producing a comparable pair

Three steps, in order:

1. Edit your manuscript in whatever you write in.
2. `dramatis ingest . --work "My Novel"` — records a new text revision and reports which files changed.
3. `dramatis analyse rev:NEW --like snap:OLD` — reads the new text with *exactly* the settings the old snapshot recorded.

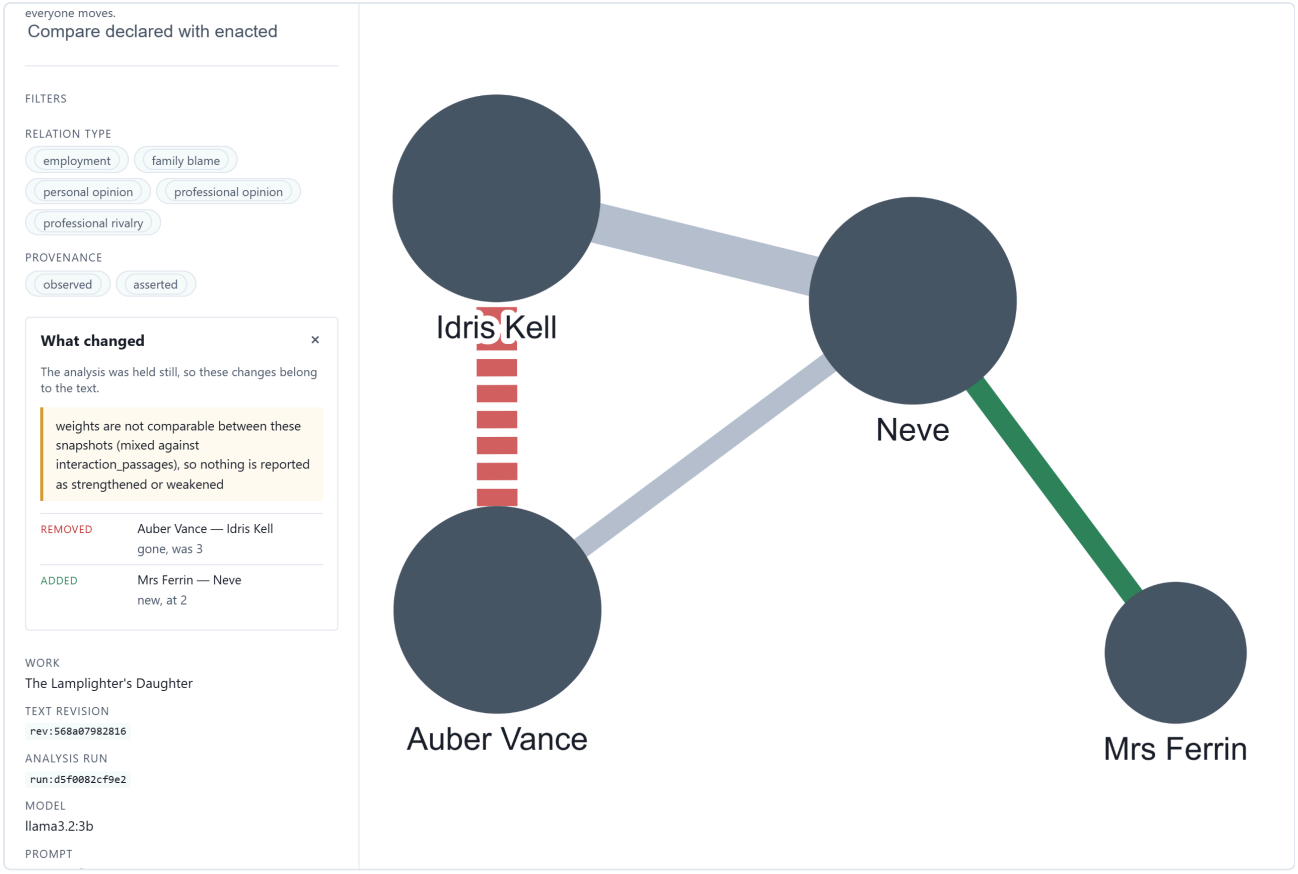
That third step is the one people skip and regret. Passing the same options by hand is not the same claim as copying what the earlier run recorded, and without it the comparison cannot separate your rewrite from a change in the reading. Dramatis prints what it copied, and if you override something explicitly it warns you that the two snapshots will no longer be comparable.

The lineage grid

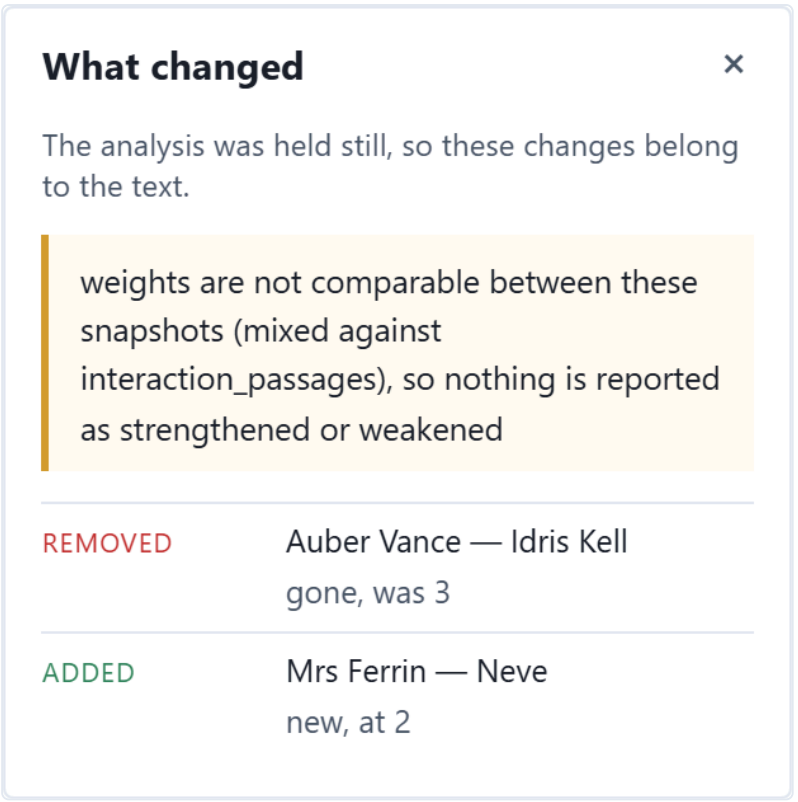


Three drafts read the same way. A row per text revision, a column per reading. Click a cell to open that snapshot; hold Shift and click a second to compare the two. A dot marks a pairing nobody has analysed, which is a different fact from an analysis that found nothing.

The comparison



The comparison as a picture. Everything either snapshot had is drawn, so what was removed is still on screen: red and dashed for gone, green for new, plain grey for unchanged. A graph cannot say “from 25 to 4”, so the same changes are also listed in words beside it.



The same comparison in words. Three things worth noticing, in the order they appear: the attribution sentence at the top; the withheld weight comparison, refused with its reason rather than computed wrongly; and only then the changes themselves.

The colours mean:

- **Added** — new in the later snapshot.
- **Removed** — gone from the later one, and still drawn, faint and dashed, because what disappeared is the half you are least able to reconstruct from memory.
- **Strengthened** — more weight behind it.
- **Weakened** — less.
- **Retyped** — the kind of relationship changed.

Attribution: the first line of the panel

Before it lists anything, the comparison says what the difference can be blamed on. There are four possible answers and you should read the sentence every time:

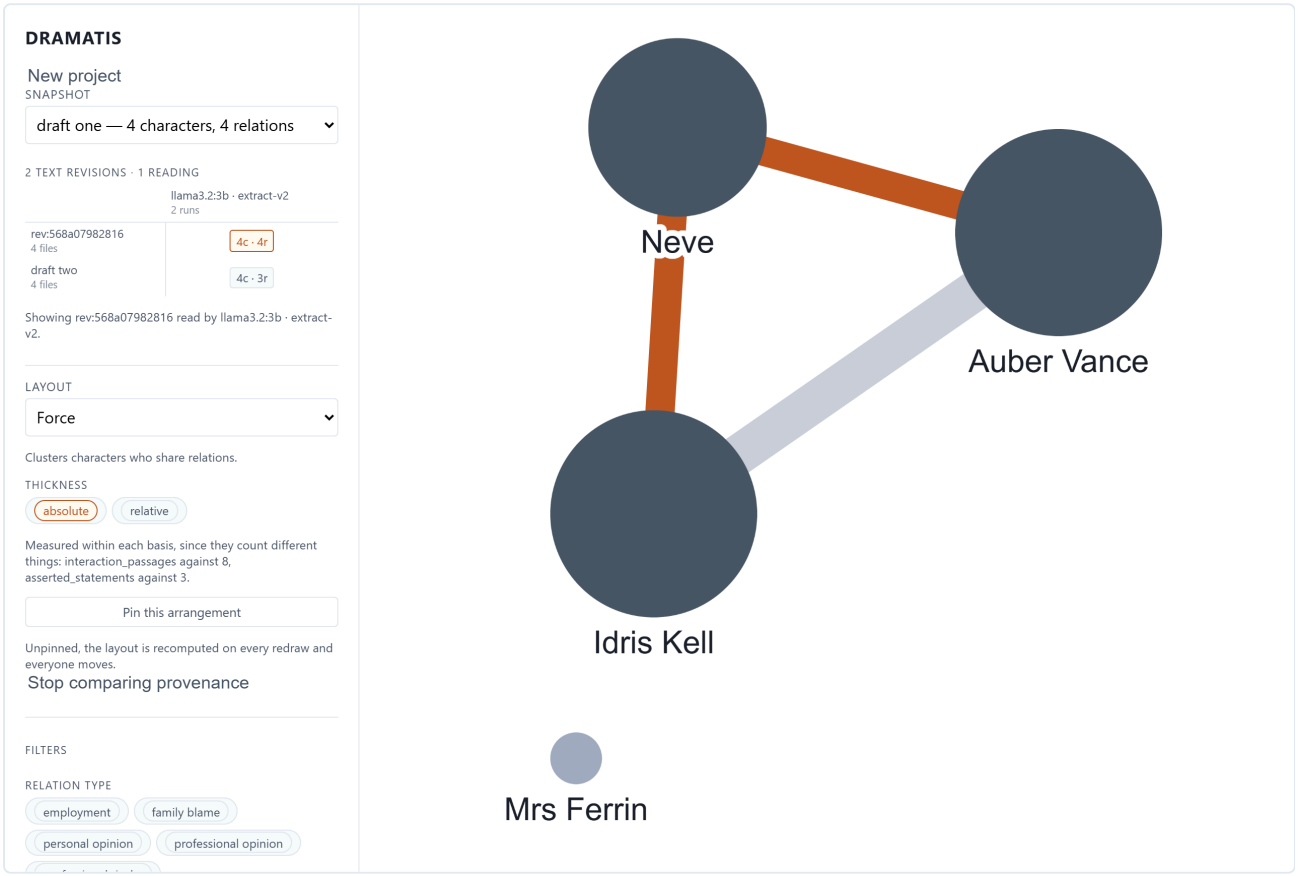
- **The work changed.** Same reading, different text. Everything below is your writing.
- **The reading changed.** Same text, different analysis. Everything below is the model, not you.
- **Both changed.** Nothing below can be credited to either, and the panel says so in a warning colour rather than quietly picking whichever moved more.
- **The edition changed.** Two editions of one work, neither superseding the other, so the difference is not something the work underwent. See *Several editions of one work* in Part IV — and note that this answer is reachable only through the interface behind the window, since the screen that picks the pair offers one work's snapshots.

Two further refusals keep the list honest. Weights are compared only where both snapshots measured on the same basis; where they disagree, the weight comparisons are withheld and the reason given. And where two characters have been merged into one, the relationships are compared through the merge, so a single act of tidying does not appear as dozens of spurious additions and removals.

21 What you declared against what you wrote

If your project contains reference material — a cast list, a series bible, an outline — Dramatis has read it separately and can put the two readings on top of each other.

The button in the sidebar reads **Compare declared with enacted**. It appears only when the snapshot has both kinds of relationship to compare, and it is never drawn at the same time as a draft comparison: two colour systems answering different questions in one picture would answer neither.



The plan against the page. Orange: two relationships the chapters carry that the cast list never mentions. Faint grey: the one relationship both agree on, receding because it needs no attention. A relationship declared and never enacted would be drawn purple and dashed; this small reading found none, which is itself a finding about the reading rather than about the book.

The two disagreements are the finding. A relationship the bible describes at length and the narrative never dramatises is a scene you meant to write; a pair who carry more of the book than anyone while the bible does not mention them is a subplot that took over while you were not looking. Both are ordinary and both are hard to see from inside the manuscript.

No weights appear anywhere in this panel, and that is deliberate. A relationship your notes state twice is not twice as strong — you repeated yourself. Passages of contact and statements of fact are different

quantities, and a column of numbers side by side would invite exactly the comparison the rest of the tool refuses to make.

Relationships that are neither declared nor enacted — the ones you corrected or entered by hand — are counted and named at the top of the panel rather than quietly excluded, so the totals here agree with the totals everywhere else.

PART IV

Curating a reading

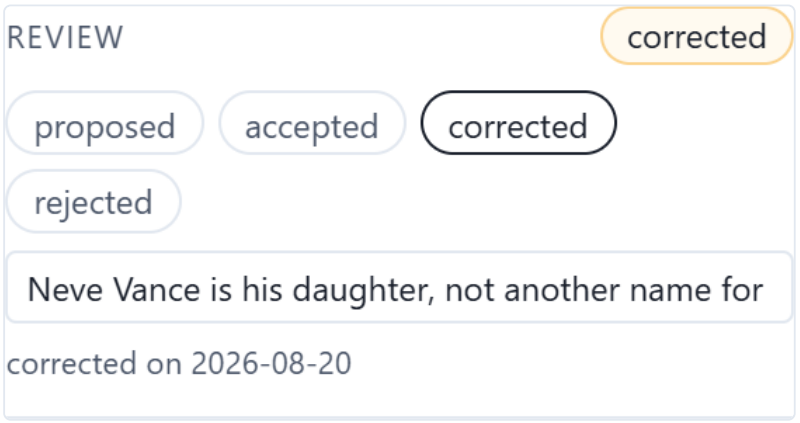
Everything a model returns is a proposal, however well evidenced. This part is about disagreeing with one and making the disagreement stick: recording what you make of a claim, putting right what a reading got wrong, settling who is who across a cast and across editions, and finding out what your corpus no longer agrees with itself about.

22 Review: what you make of a claim

A reading proposes. Until somebody has looked at it, that is all it is. Review is the record of somebody having looked.

Every character and every relationship carries one of four states:

`proposed`	Nobody has ruled on it. Everything starts here.
`accepted`	You have looked at it and it is right.
`corrected`	It was wrong and you have said what it should be. A reason is required.
`rejected`	It should not be in the graph at all.



The review control, above the claim it is about. The reason box is offered always and required only for *corrected*: a correction that does not say what it corrects is indistinguishable from a rejection somebody softened. Why the button is unavailable is written out rather than hidden in a tooltip, because a disabled control does not reliably receive a hover.

From the command line

```
dramatis review
```

```
snap:243d20a2f047  The Lamplighter's Daughter
7 subjects - 7 proposed; 0 ruled on by a person

  [proposed ] character  Auber Vance   char:auber-vance
  [proposed ] character  Idris Kell   char:idris-kell
  [proposed ] character  Mrs Ferrin   char:mrs-ferrin
  [proposed ] character  Neve   char:neve
  [proposed ] relation   Idris Kell -- Neve   rel:idris-kell--neve
  [proposed ] relation   Auber Vance -- Neve   rel:auber-vance--neve
  [proposed ] relation   Mrs Ferrin -- Neve   rel:mrs-ferrin--neve
```

Rule on one of them:

```
dramatis review --relation rel:auber-vance--neve --status accepted
dramatis review --character char:mrs-ferrin --status rejected \
    --note "a walk-on; not a member of the cast"
```

```
relation rel:auber-vance--neve is now accepted
character char:mrs-ferrin is now rejected
  note: a walk-on; not a member of the cast
```

And come back to what is left:

```
dramatis review --pending
```

```
7 subjects - 5 proposed, 1 accepted, 1 rejected; 2 ruled on by a person
```

Four things the design guarantees

- **The snapshot is not touched.** Decisions live beside it. What the snapshot says is what the analysis proposed; what review says is what a person has since decided, and a reader is entitled to see those apart.
- **A decision is about a claim, not about a document.** It is keyed to the character or relationship, so it survives re-analysis. You are never asked to re-accept a cast you have already been through — which is how a review tool stops being used.
- **Nothing is overwritten.** Each decision is appended and the newest stands. That somebody once accepted what has since been rejected is part of the record. `--history` shows it:

```
dramatis review --character char:mrs-ferrin --history
```

```
character char:mrs-ferrin
  2026-08-20T20:16:59+00:00  rejected  in snap:243d20a2f047
      a walk-on; not a member of the cast
```

- **A claim that was never made cannot be reviewed.** Ruling on an identifier no reading ever proposed is refused, because the usual cause is a typo rather than a considered act.

23 Correcting what a reading got wrong

Review lets you say *this is wrong*. Correction lets you say *this is what it should be* — and makes the answer stick across every reading that follows.

This is the single most useful thing in Part IV, because the alternative is making the same fix after every analysis until you stop bothering.

What may be corrected

ON A CHARACTER	ON A RELATIONSHIP
`name`, `kind`, `aliases`, `notes`	`types`, `valence`, `directed`, `notes`

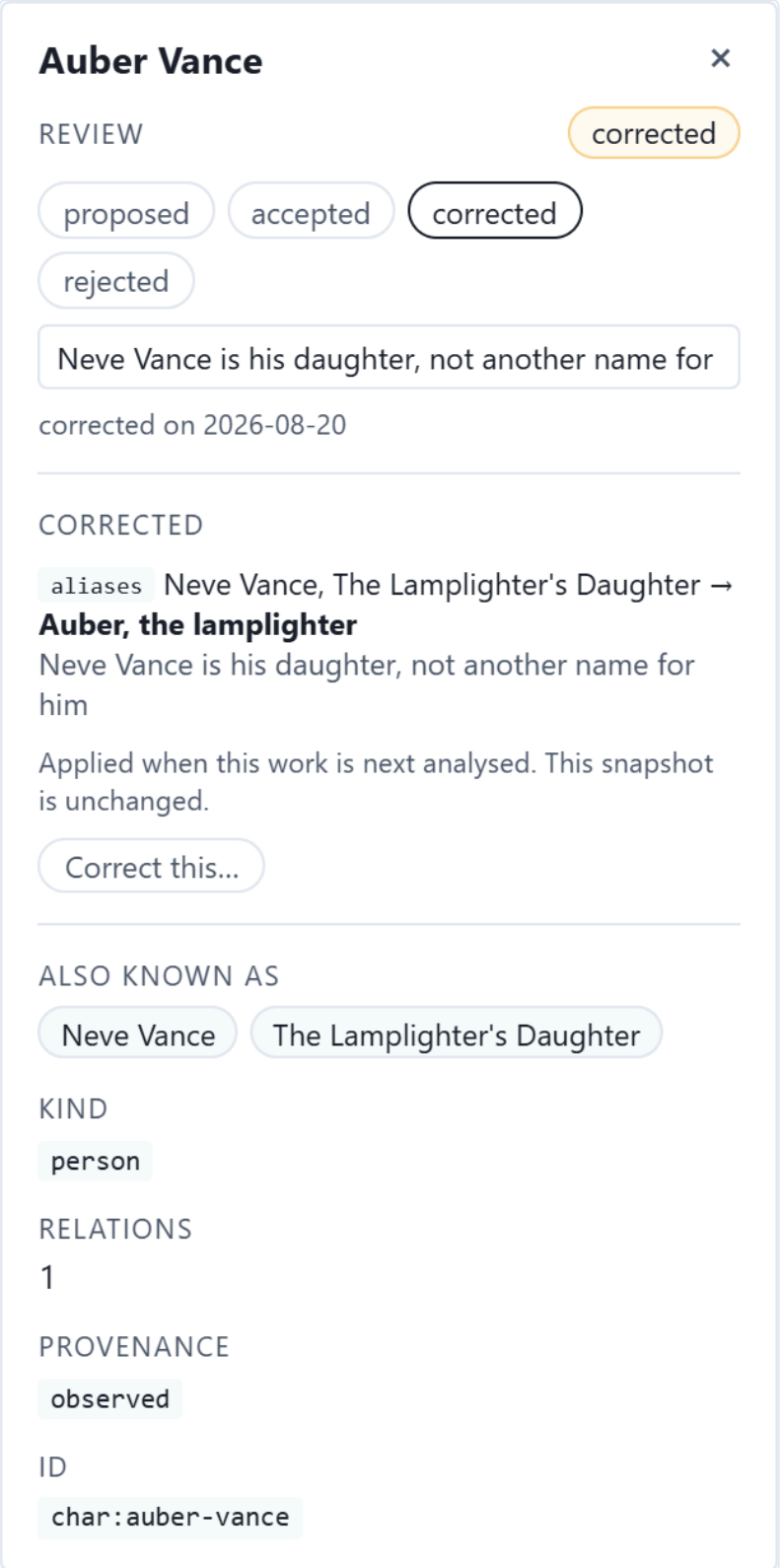
Three things are deliberately not correctable, and each refusal names its reason if you try. **A weight** is a count on a declared basis, not an opinion. **Evidence** is verified against your text or rejected, so there is nothing to overrule. **Identity** — whether two characters are one person — is the next chapter's business, because it works differently.

Making one

```
dramatis correct --character char:auber-vance --field aliases \
    --value "Auber" "the lamplighter" \
    --note "Neve Vance is his daughter, not another name for him"
```

```
corrected character char:auber-vance
  aliases: ["Neve Vance", "The Lamplighter's Daughter"] -> ["Auber", "the lamplighter"]
  applies to the next analysis; this snapshot is unchanged
```

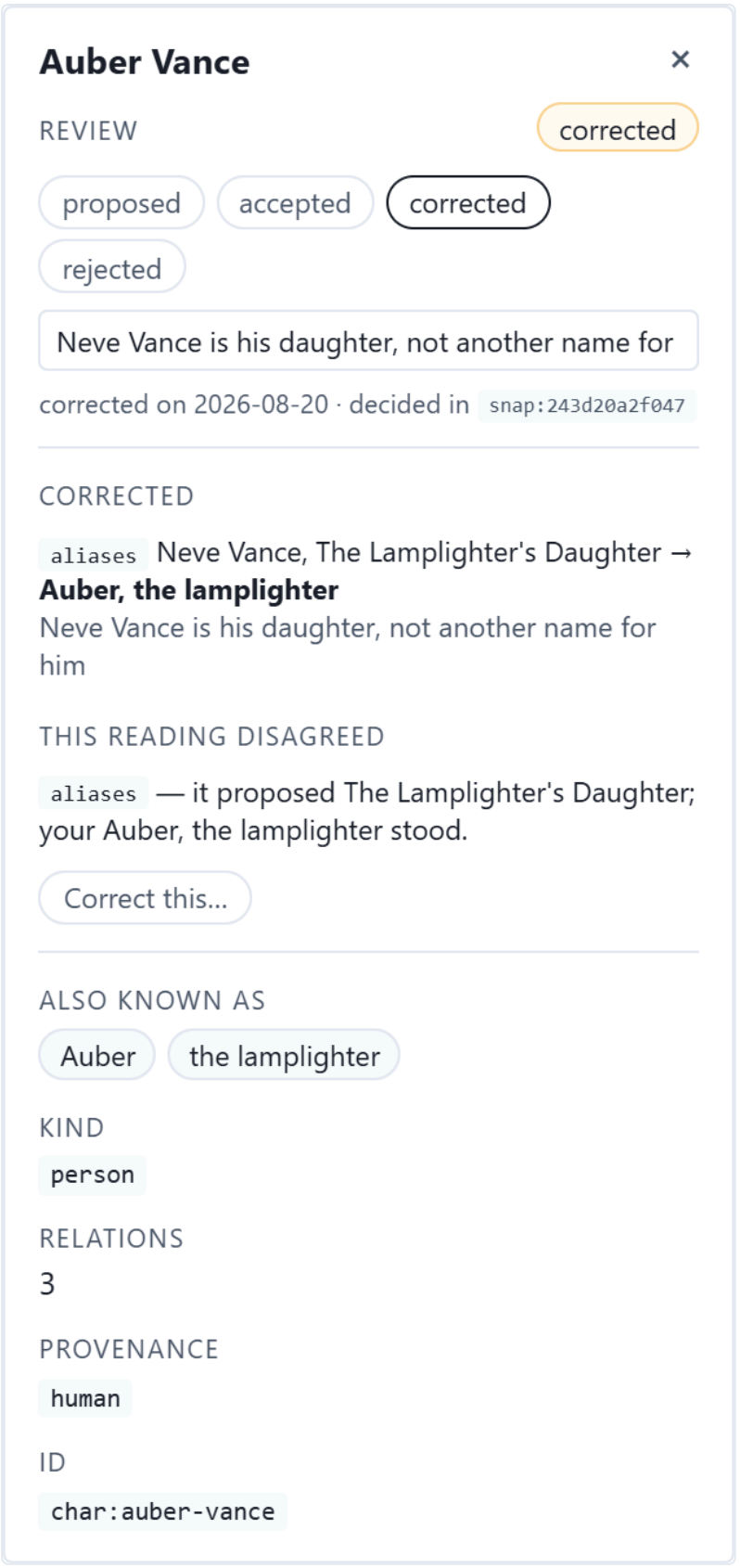
Several words after `--value` make a list, which is what `aliases` and `types` want. In the browser the same thing is a *Correct this…* button under the review control; the box starts at what the reading currently says, so a correction is an edit rather than a retyping.



A correction, in the reading it was made against. Read the panel downwards: the review status is now *corrected*; the correction itself is listed with what it replaced; and then the line that catches everybody out — the snapshot is unchanged, so *Also known as* still shows the old aliases. That is not a bug. This snapshot is what the analysis said, permanently.

Where it lands

A correction is written into the graph when the *next* snapshot is built. Analyse the work again and it appears — as does the consequence the design requires, that a corrected node or edge becomes `human`:



The same character, one analysis later. *Also known as* now carries the corrected names, provenance has become `human`, and a new block has appeared: *this reading disagreed*. The model proposed something else; your correction stood; the disagreement is on the record rather than swallowed.

The analysis says the same thing as it runs:

```
note  this reading gives character char:auber-vance aliases=["The Lamplighter's Daughter"],
      but a correction holds ["Auber", "the lamplighter"]. The correction stands.
```

Why it works that way

- **A person outranks a run.** Where a later reading disagrees with a correction, the correction wins. The alternative throws away your work every time a model is run.
- **But the run is never silently ignored.** Its competing claim is recorded as a conflict and shown. A model that has changed its mind is worth knowing about — it may be right, and then you correct your correction.
- **A corrected relationship leaves the declared-against-enacted comparison.** It counts as entered by hand, because your edit is not evidence about the corpus and pretending otherwise would corrupt the one finding that view exists for.

Everything you have ever put right is on the record:

```
dramatis correct --character char:auber-vance --history
```

24 One person, two names: merge and split

The commonest fault in any character graph is one person appearing twice. This is the fix, and it works differently from a correction: it changes how the *next* reading resolves names, rather than what this one said.

Alias resolution can create a character and attach a new name to an existing one. It cannot decide that two characters it already knows are the same person, because merging is destructive and cannot be reviewed after the fact. So it stays a human act.

Merging

```
dramatis merge char:neve-vance --into char:neve --note "one person, two names"
```

```
merged char:neve-vance into char:neve
  Neve now answers to Neve, Neve Vance
  registry now holds 4 character(s)
  the next analysis of this collection reads it this way
```

The absorbed character gives up its names and is retired — kept, not deleted, so that an identifier in an older snapshot can still be traced back to something. Everything you had already recorded against it, every review and every correction, follows through the merge onto the character that survived.

Splitting

The same shape in reverse: move some names onto a new character.

```
dramatis split char:auber-vance --form "Neve Vance" --name "Neve Vance" \
    --note "the daughter, wrongly folded into her father"
```

```
split char:neve-vance out of char:auber-vance
  Neve Vance answers to Neve Vance
  Auber Vance answers to Auber Vance, The Lamplighter's Daughter
  registry now holds 5 character(s)
  the next analysis of this collection reads it this way
```

Repeat `--form` for several names. At least one must stay behind: moving every name is a rename, not a split, and Dramatis will say so. Because both operations are the same shape — surface forms moving between characters — a split is also the only way to undo a merge.

The two-step that fixes a folded character

Where a reading has folded two people into one, the repair is a split followed by a merge: separate the wrong names out, then join them to the character they belong to.

```
dramatis split char:auber-vance --form "Neve Vance" --name "Neve Vance"
dramatis merge char:neve-vance --into char:neve
```

> CORRECTION AND IDENTITY ARE DIFFERENT TOOLS
>
> A correction changes what the graph *says* about a character. A merge or split changes who the next reading thinks that character *is*. Both are often wanted together, and Dramatis notices when they interact: split a name away from a character whose aliases you have already corrected and it warns you that the correction still governs that list until you update it too.

Every decision is on the record, and `dramatis characters` prints it:

```
decisions   2
  2026-08-20T20:17:37+00:00  split char:auber-vance => char:neve-vance  (Neve Vance)
  2026-08-20T20:17:37+00:00  merge char:neve-vance -> char:neve  (Neve Vance)
```

25 The continuity report

A revision changes the text under an analysis made of the text as it was. Most of what that breaks is harmless and fixes itself when you re-read. Three things do not, because re-reading cannot see them.

```
dramatis continuity
```

A name the work has moved on from

You rename somebody in the chapter you are working on and miss the two other places that mention them. Re-analysing does not report this — it reports a cast, and a cast with a stale name in it looks exactly like a cast. What *is* checkable is that a name the last reading found in a document is no longer written there and is still written somewhere else:

```
The Lamplighter's Daughter  (snap:243d20a2f047)
  reading of rev:3fbe3124d583 against rev:ba710042c22a
  1 finding(s)

names the work has moved on from
  Idris Kell: 'Kell' is gone from chapter-03.md, but still appears 5 time(s) elsewhere
    cast.md: ...oud of her and cannot say so. - **Idris [Kell]** - the inspector who cut his
round in...
    cast.md: ...her father. She manages him. - **Idris [Kell]** - she finds him reasonable, which
she...
    cast.md: ...she has not told her father. ## Idris [Kell] Borough inspector. Efficient,
unimagin...
    chapter-02.md: ...s name?" Mrs Ferrin found the notice. "[Kell]. Inspector Idris Kell." Neve
wrote it...
    chapter-02.md: ...ound the notice. "Kell. Inspector Idris [Kell]." Neve wrote it down, which
surprised...
```

Every remaining place is quoted with its surroundings, so the fix is a list rather than a search. The comparison is between whole documents on purpose: a rename is a find-and-replace in the file you were working on, and what it misses is another file. The honest cost is that a work kept in a single file can never produce this finding, because there is no elsewhere for a name to be stale in.

A citation with nowhere to land

Evidence records a structural position as well as a quotation. Delete a section and the position stops existing, so a citation that used to open the source opens nothing. The report says which claims point at a place the work no longer has, and whether their words survive anywhere.

A draft you replaced and kept

If your structure map says one document revises another and the revision holds both, every scene in that chapter is being read twice and every interaction in it weighted double. Nothing else notices: to a reader that was not told they are the same chapter, two copies are a perfectly ordinary corpus.

> IT REPORTS; IT NEVER REPAIRS
>
> Every one of these has more than one right answer. A stale name might want fixing in the old chapters or reverting in the new one; a duplicated draft might want deleting or might be deliberate. Choosing is the author's job, so `continuity` changes nothing, calls no model, and reaches no network.

26 A cast that spans several works

When a project holds more than one work, the shared character registry becomes useful in its own right:

```
dramatis characters
```

```
Jane Austen - 102 characters across 1 works

  Caroline Bingley  [person]
    also        Caroline, Miss Bingley
    appears in  Pride and Prejudice (12 relations)

  Charlotte Lucas  [person]
    also        Charlotte, Miss Lucas, Mrs. Collins
    appears in  Pride and Prejudice (13 relations)
```

Whoever spans the most works is listed first, because that is the question a shared-universe registry is opened to ask. To see only those:

```
dramatis characters --spanning
```

Two details worth knowing. Appearances are worked out from snapshots rather than stored separately, so they cannot fall out of step — and only the *newest* snapshot of each work is consulted, because a character you cut in the second draft does not appear in that novel any more. A character present in the registry but in no current reading is listed as such rather than left blank, since that is a real state and a blank line reads as a bug. This command calls no model and reaches no network.

27 Several editions of one work

A draft supersedes the draft before it. An *edition* supersedes nothing. A first edition and the author's revised edition are both authoritative, both citable and both current, and a scholar may legitimately want the graph of the earlier text specifically. Treating the later one as simply the newest revision would answer a question nobody asked and destroy the one they did.

So Dramatis makes the edition part of the work's identity. Name one when you ingest:

```
dramatis ingest editions/1889-first/text.md \
    --work "The Salt Road" --edition 1889-first --label "first edition"

dramatis ingest editions/1903-revised/text.md \
    --work "The Salt Road" --edition 1903-revised --label "revised edition"
```

```
ingested: rev:f0ffdac2ff9e (1,207 characters, sha256 f0ffdac2ff9e...)
  store       salt-road.sqlite  (new project)
  collection  col:the-salt-road
  work        work:the-salt-road@1889-first
  document    doc:text-md-f99c3a1abac6
  revision    rev:f0ffdac2ff9e
```

The edition rides in the work identifier after an `@`, so the two ingests land on two works rather than on one work with two revisions:

```
dramatis status
```

```
collection  The Salt Road  (col:the-salt-road)
registry    4 character(s)

work        The Salt Road  [1889-first]  (work:the-salt-road@1889-first)
            1 revision(s), 1 snapshot(s)

work        The Salt Road  [1903-revised]  (work:the-salt-road@1903-revised)
            1 revision(s), 1 snapshot(s)
```

Each is analysed on its own, compared on its own, and cited on its own. Nothing about a project that never names an edition changes: a work ingested without `--edition` keeps the plain `work:the-salt-road` it has always had.

One cast, two texts

Both editions sit in one collection, which is the whole reason to put them there: they share one registry, so a character whose name did not change between editions is *literally the same character*, with nothing for you to declare. Corin Ashe is Corin Ashe in both.

The exception is the character who was renamed. In *The Salt Road* the confidante is Hesper in 1889 and Perdita in 1903 — one person, two surface forms, and no text in which both occur.

```
dramatis characters
```

```
The Salt Road - 4 characters across 2 works

  Corin Ashe  [person]
    also        Corin
    appears in  The Salt Road [1889-first] (2 relations), The Salt Road [1903-revised] (2
relations)

  Marlow  [person]
    appears in  The Salt Road [1889-first] (2 relations), The Salt Road [1903-revised] (2
relations)

  Hesper  [person]
    appears in  The Salt Road [1889-first] (2 relations)

  Perdita  [person]
    appears in  The Salt Road [1903-revised] (2 relations)
```

Each appearance is named with its edition, here and in `status` and in the `--json` output, for one reason: every title in this listing has a number beside it, and two lines differing only in an identifier suffix are how somebody reads one edition's relation count as the other's.

Correspondence, which is not a merge

Hesper and Perdita are one figure. The obvious move is `dramatis merge`, and it is the wrong one — decisively so. A merge moves the surface forms, so the next reading of the 1903 text would resolve “Perdita” to Hesper and the 1903 graph would show a node captioned with a name that occurs nowhere in the 1903 text. That is not a tidier registry. It is a false statement about a published edition.

What you want instead is a correspondence:

```
dramatis correspond char:hesper char:perdita --note "renamed in the revised edition"
```

```
note: neither character was changed. Each edition's graph still names the character that edition
names.
Hesper and Perdita are one figure across editions
```

That first line is the entire design. Nothing moved, nothing was retired, and every snapshot already written says exactly what it said before. What the record does is give the comparison a way to see the two as one person. With no arguments the command lists what has been declared; `--withdraw` takes one back.

```
dramatis correspond
```

```
1 correspondence(s)
  Hesper = Perdita across editions (renamed in the revised edition)
```

Two characters who appear in the same text are refused, and the refusal names the command that is wanted instead:

```
error: Corin Ashe and Marlow both appear in work:the-salt-road@1889-first,
work:the-salt-road@1903-revised, so this is not a cross-edition correspondence. Two
characters in one text who are the same person is a merge: `dramatis merge` moves the
surface forms and retires one.
```

Comparing two editions

A comparison across editions reports its attribution as **edition** — its own answer, neither "the text changed" nor "the reading changed". The distinction is the point: *text* would tell you the 1903 reading is a change the work underwent and that the earlier state has been left behind, which is the opposite of the truth about two editions. The pair of editions compared is carried alongside, because which two they were is part of the citation.

Where you have declared a correspondence, the renamed character contributes a single *renamed* entry and none of her relationships move. Where you have not, the comparison counts the characters sitting on one side only and points at `correspond` — a hint rather than a fault, because some of them genuinely are in one edition and not in the other, and from inside a comparison the two cases look identical.

> NOT REACHABLE FROM THE WINDOW YET
>
> The comparison screen offers the snapshots of *one* work, because that is what the lineage grid is — a work's two time axes. Two editions are two works, so there is no way to pick the pair in the browser. The interface behind the window will compare them, and a command-line `diff` is on the list of small things not yet built. Until one of the two arrives, edition comparison is something Dramatis can do and cannot yet be asked to do conveniently.

> THREE EDITIONS
>
> A correspondence is a pair. A figure renamed twice, across three editions, maps to one of the two counterparts rather than to a chain. Two is what the design handles today, and this is a limitation rather than a decision.

28 Every command, at a glance

COMMAND	WHAT IT DOES	MODEL?
`dramatis status`	Which project file is in use, how it was found, and everything in it: settings, collection, works, revisions, snapshots.	No
`dramatis structure`	What a corpus appears to hold, with the evidence for each guess. `--set` corrects, `--confirm` saves, `--forget` discards, `--ask` has a model read the documents, `--drive` points at a Drive folder.	Only with `--ask`
`dramatis authorise`	Consent once, in a browser, to read-only access to Google Drive. `--status` reports, `--forget` deletes the local credential.	No *(reaches Google)*

`dramatis ingest`	Reads a file, a folder, or a Drive folder into the project as a text revision. The only command that creates a project.	No *(reaches Drive only with `--drive`)*
`dramatis analyse`	Reads a stored revision with a model, verifies every quotation, applies your standing corrections and registry decisions, and records a snapshot. *(`analyze` also works.)*	**Yes**
`dramatis serve`	Opens the project in your browser on this machine. Default address `127.0.0.1:7373`.	No
`dramatis review`	Where review stands, and how to move it. `--status` records a decision, `--pending` lists what is unruled, `--history` shows every decision ever taken.	No
`dramatis correct`	Put right what a reading got wrong, so it survives re-analysis. `--history` lists everything ever corrected.	No
`dramatis merge`	Declare that two registered characters are one person.	No
`dramatis split`	Declare that one registered character is two people.	No
`dramatis correspond`	Declare that two characters in two editions are one figure. Changes neither of them. With no arguments, lists what has been declared; `--withdraw` takes one back.	No
`dramatis continuity`	What the corpus no longer agrees with itself about. Reports; never repairs.	No
`dramatis characters`	The collection's cast, which works each character appears in, and the identity decisions you have taken.	No
`dramatis export`	Write a reading out for another tool: `graphml`, `gexf`, `csv`, `jsonld`, `annotations` for the evidence, or `snapshot` for the stored document itself. `-o` names the file; without it, most formats go to the screen.	No
`dramatis import`	Read a Dramatis document — yours or another tool's — into this project. Refuses before it writes if anything in it collides.	No
`dramatis validate`	Checks a snapshot file against the published schema and its internal references. Works on documents produced by other tools.	No

Options shared by most commands:

`--store PATH`	Name the project file directly instead of letting Dramatis search for it. Also accepts a PostgreSQL address for shared installations.
`--json`	Machine-readable output on standard output. Every remark, note and warning goes to standard error instead, so the JSON is never contaminated.
`--note`	On `review`, `correct`, `merge`, `split` and `correspond`: why, for whoever reads this later. Required when correcting.
`--snapshot ID`	On `export`: which reading to write out. Without it, the newest in the project, and the command says which one it chose.
`--help`	Full options for any command: `dramatis analyse --help`. The help text is written in the same voice as this manual.

PART V

Taking it elsewhere

A reading that can only be seen inside Dramatis is not much use to a discipline that argues in journals and lays its networks out in Gephi. Three chapters: writing a reading out, publishing the evidence under it, and reading in a document somebody else produced. All three are command-line work, all three call no model, and none of them touches the network.

29 Exporting a reading

One command, one format name, one file:

```
dramatis export gexf -o pride-and-prejudice
```

```
note: exporting snap:9204c78ca953, the newest reading here
wrote pride-and-prejudice.gexf
```

Without `--snapshot` it takes the newest reading in the project and tells you which one that was, so a file you come across on your disk a month later is traceable. The extension is added for you. Without `-o` the export goes to the screen, which is convenient for a quick look and impossible for CSV — CSV is two files, so it insists on a name.

FORMAT	WHAT IT IS FOR	WRITES
`graphml`	The richest of the four. Gephi, Cytoscape, yEd, NetworkX. Carries every scrap of provenance the reading has, on the graph itself.	`NAME.graphml`
`gexf`	Gephi's own format, in the 1.2draft version every Gephi in use actually reads.	`NAME.gexf`
`csv`	A node list and an edge list, for a spreadsheet or anything that eats tables.	`NAME.nodes.csv`, `NAME.edges.csv`
`jsonld`	Linked data, for a triple store or anything that speaks JSON-LD.	`NAME.jsonld`
`annotations`	The evidence, as W3C Web Annotation. The next chapter.	`NAME.annotations.jsonld`
`snapshot`	The stored document itself, unchanged. What `import` reads, and the other half of a round trip.	`NAME.dramatis.json`

What travels with the graph

Every node carries its label, kind, aliases, salience, provenance, review status, and a count of the evidence behind it. Every edge carries its weight and — always, in every format — the **basis** of that weight. Each of these formats has a native notion of an edge weight and none has a native notion of what the number counts, so a weight travelling without its basis is a number waiting to be compared with something it is not comparable to.

What differs between the four is how much room each has for the reading *as a whole*. Rather than level down to the poorest, each carries what it can:

- **GraphML** takes the lot, on the graph element: work, edition, collection, text revision and its fingerprint, analysis run, model, provider, prompt version and hash, schema version.
- **GEXF** has only a fixed `<meta>` block, so it carries one citation line naming both time axes. Gephi shows it on import.
- **CSV** has nowhere at all — which is exactly why a pair of lists in somebody's spreadsheet is the artefact that gets separated from its context. So the snapshot identifier is repeated on *every row*: redundant, and the only field that survives a sheet being loaded, edited and saved again.
- **JSON-LD** carries everything, as linked data.

```
snapshot_id,id,source,target,weight,weight_basis,directed,types,...
snap:9204c78ca953,rel:elizabeth-bennet--mr-fitzwilliam-darcy,char:elizabeth-bennet,
char:mr-fitzwilliam-darcy,46,interaction_passages,false,,...
```

> YOUR REVIEW DECISIONS ARE APPLIED ON THE WAY OUT
>
> An export is the copy that gets cited, so it should say what you now think rather than what the model first proposed. Every claim you accepted, corrected or rejected leaves with the status you gave it, read over the stored document. The exception is `snapshot`, which writes the stored document exactly as it stands — that one is not a copy to be cited but the artefact itself, and a round trip that rewrote a field on the way out would have stopped meaning anything.

Where the identifiers point

In the JSON-LD export every identifier is already a compact IRI, because Dramatis has always namespaced them by kind — `char:`, `rel:`, `work:`, `rev:`, `run:`, `snap:`, `doc:`, `col:`. The context has only to say what each prefix expands to; nothing is rewritten.

One distinction there is worth understanding if you deposit the file anywhere two exports might meet. A *content-derived* identifier means the same thing everywhere: `rev:e3bb81d19b34` is that exact text in your store and in anybody's. A *name-derived* one means something only inside the registry that minted it: your `char:mary` is one person and somebody else's is another. So revisions, runs, snapshots and documents expand globally, and works, characters and relations expand underneath their collection — otherwise every Mary in the world would become one node the moment two exports met in a triple store.

The context is written into the file rather than fetched from a web address, deliberately: an export is the thing that must still make sense when nothing is reachable.

NO DOWNLOAD BUTTON

`dramatis export` is a command. The browser window has no export control yet, so exporting is something you do in the terminal beside it.

30 Exporting the evidence

The four graph formats carry a *count* of the evidence behind each claim rather than the evidence itself, and that is deliberate. A piece of Dramatis evidence is a locator — an ordered path of typed segments — plus a quotation with the text either side of it, and there is no honest way to put that in a spreadsheet cell. Flattening it would have meant shipping a lossy encoding that people would find first.

So the evidence has its own export, in a format built for exactly this shape:

```
dramatis export annotations -o pride-and-prejudice
```

```
wrote pride-and-prejudice.annotations.jsonld
```

The result is a W3C Web Annotation `AnnotationCollection`, labelled with which reading it came from, holding one annotation per quotation. This is the only export that contains a word of your text, and the only one another scholar's tooling can consume without knowing what Dramatis is.

```
{
  "id": ".../annotation/dacb5d7a13b1dca2",
  "type": "Annotation",
  "motivation": "describing",
  "body": [
    { "id": ".../rel/elizabeth-bennet--mr-fitzwilliam-darcy",
      "type": "dramatis:Relation",
      "label": "Elizabeth Bennet -- Mr. Fitzwilliam Darcy" },
    { "type": "TextualBody", "purpose": "commenting",
      "value": "refusal transforms Darcy" }
  ],
  "target": {
    "type": "SpecificResource",
    "source": { "id": ".../doc/pride-and-prejudice", "type": "Text",
                "dramatis:sha256": "e3bb81d19b34..." },
    "selector": {
      "type": "TextQuoteSelector",
      "exact": "the shock of Elizabeth’s scornful refusal",
      "prefix": "restoration to healthy conditions than",
      "suffix": "acting on a nature_ ex hypothesi _gener"
    },
    "dramatis:locator": [ { "type": "section", "index": 30 } ],
    "dramatis:matching": "whitespace-collapsed"
  },
  "dramatis:provenance": "observed",
  "dramatis:reviewStatus": "proposed"
}
```

The `prefix` and `suffix` are what let a consumer find the passage again in a text that has been reformatted since — the same mechanism Dramatis uses to re-anchor a quotation after you have edited around it.

> THE LINE THAT KEEPS THE EVIDENCE FROM LOOKING INVENTED
>
> Every target says `"dramatis:matching": "whitespace-collapsed"`. Dramatis defines *verbatim* against text whose runs of whitespace have been collapsed, because a quotation in a hard-wrapped novel almost always crosses a line break. A consumer searching for a byte-exact match would fail on most of a novel and conclude the evidence had been fabricated. The Web Annotation vocabulary has no term for this, so Dramatis states it in its own — on every target rather than once at the top, because an annotation that gets separated from its collection has to carry the instruction with it.

31 Reading in someone else's document

A Dramatis snapshot is a document conforming to a published schema, which is a claim worth nothing until something reads one back. `dramatis import` is that something, and it does not care which tool wrote the file:

```
dramatis import their-reading.dramatis.json
```

```
note: 1 document(s) recorded without their text. A Dramatis document carries
hashes, not the work. Quotations are readable; the passages around them need
`dramatis ingest` of the same files.
imported snap:9204c78ca953: 102 character(s), 241 relation(s), 1022 piece(s) of evidence
```

What arrives behaves like a reading of your own: it opens, it compares, it exports, its cast is registered in your collection and its surface forms are claimed. Importing the same file twice changes nothing, and says so.

A document brings a graph, not the work

That first note is the one thing to understand about the format, and it is a feature rather than a shortfall. A Dramatis document carries a fingerprint for each source document and never a word of the text. That is what makes one safe to send: it says what was read without shipping somebody's unpublished manuscript.

The consequence is that an imported reading has quotations you can read — they are inside the document — but no passage around them to open, because that needs the source. It is not permanent. A document's identifier is derived from its path and its content, so if you ingest the same files the text lands on the same identifier and joins the imported graph with nothing to reconcile. The empty document is a row waiting for its text, not a wrong answer.

It refuses before it writes

Everything is checked first — a snapshot identifier already used for different content, a character identifier already meaning somebody else, a surface form already claimed — and if anything collides, nothing at all is written:

```
error: char:elizabeth-bennet is already in this project as 'Elizabeth Bennet',
and the document calls it 'Eliza Bennet-Smith'. Two different people cannot share
an identifier; if they are the same person, say so with `dramatis merge`.
```

Half an imported reading would be worse than none, because the half that landed looks exactly like a reading somebody meant to have. And identity is refused rather than guessed: deciding that two records are the same person is a judgement you record with `merge`, not one an importer should take on your behalf by reading a file.

To check a document before importing it — or check one you are about to send — `dramatis validate` reads it against the schema and its internal references without touching your project.

The round trip

The `snapshot` export writes the stored document as it stands, so a reading can leave and come back unchanged:

```
dramatis export snapshot -o my-reading
dramatis import my-reading.dramatis.json --store elsewhere.sqlite
```

This is also how you hand a colleague your graph *without* handing them your manuscript. *Sharing*, in Part VII, is where that difference matters.

PART VI

A project over time

One short novel, followed from an empty folder to a published study, in twelve milestones. Each gives the reason for the step, the exact commands, and what you should look at afterwards. Follow it with your own work substituted and you will have used every feature Dramatis currently has.

32 Twelve milestones with one short novel

The Lamplighter's Daughter. Auber Vance has lit the same round for forty years. Neve, his daughter, keeps the accounts at a chandlery. Idris Kell is the borough inspector who cuts the round in half. There is a cast list. There will be three drafts.

> ABOUT THE SCREENSHOTS IN THIS PART
>
> These readings were produced with a small model running locally (`llama3.2:3b`) so that the whole example could be reproduced free of charge and with nothing leaving the machine. That has a visible consequence, called out at milestone 3, and it is left in rather than tidied away — because milestones 9 and 10 are about fixing exactly that kind of mistake.

MILESTONE 1 A folder, looked at

Why. Before spending anything, find out what Dramatis thinks your folder holds. This costs nothing and reaches nothing.

```
cd ~/writing/lamplighter
ls
```

```
cast.md    chapter-01.md    chapter-02.md    chapter-03.md
```

```
dramatis structure .
```

> WHAT TO LOOK AT
>
> Four documents, every role *unknown*, and under each the reason: "needs the text read, not the folder named". Dramatis will not guess that a file called `cast.md` is a cast list. That refusal is the feature — your filing habits are yours, and a wrong guess here would quietly poison every graph that followed.

MILESTONE 2 The project exists

Why. Answer the role question once, and read the text in. Still no model, still no cost.

```
dramatis serve --store dramatis.sqlite
```

Open `http://127.0.0.1:7373`. The New project panel is already open because there is nothing to show. Type the folder path, press *Read it*, mark `cast.md` as **reference** and the three chapters as **narrative**, give the work a title, and press *Create the project*.

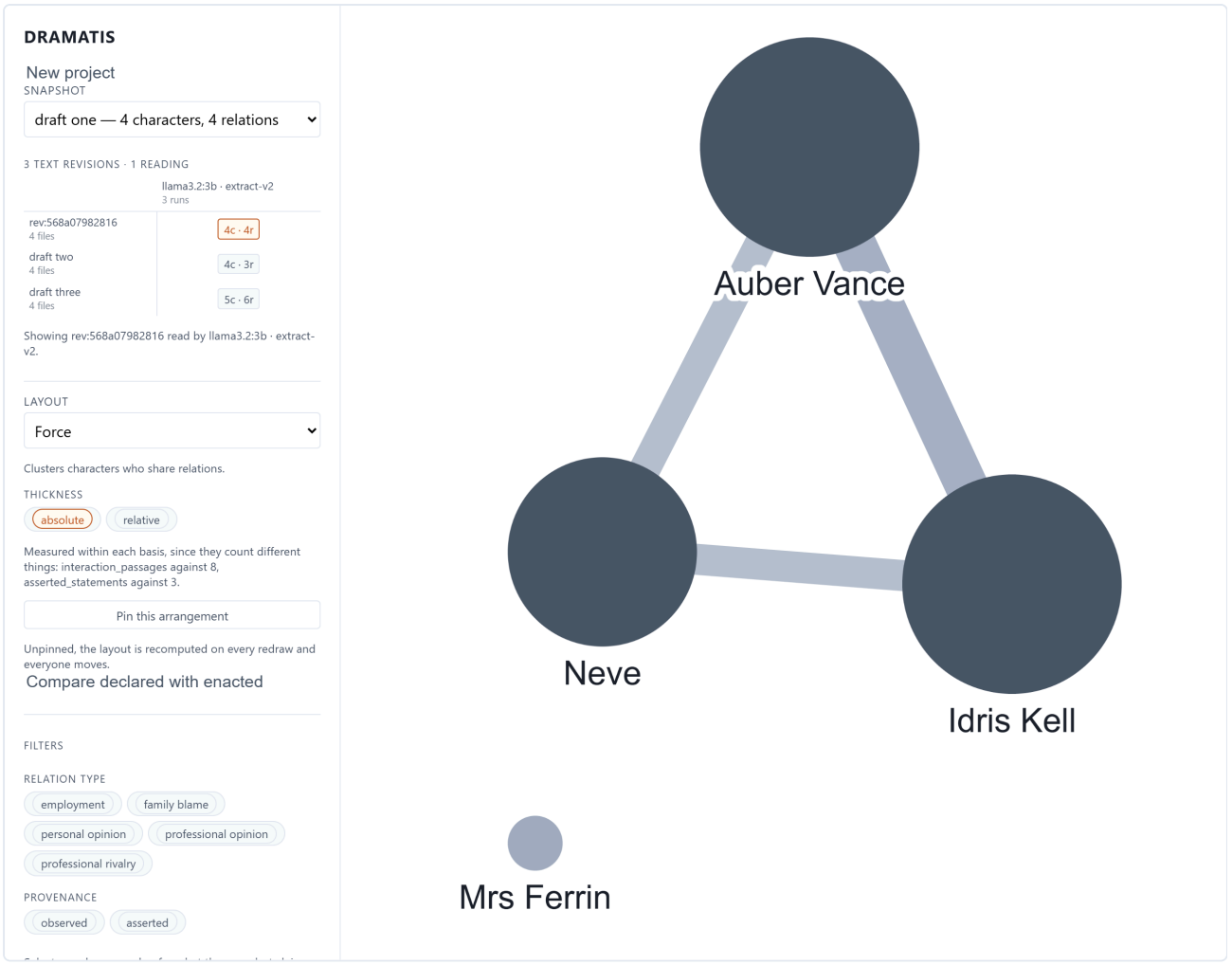
> **WHAT TO LOOK AT**
> "Ready: 4 documents, 1 as reference material." A file called `dramatis.sqlite` now sits beside your chapters. That single file is the project — the text, the settings, and every reading and decision you will ever make.

MILESTONE 3 The first reading

Why. Now spend something. This is the first and only step that calls a model.

```
dramatis status                       # find the revision identifier
dramatis analyse rev:568a07982816 --provider ollama --label "draft one"
```

```
snapshot    snap:bb9660a6fe4b
  characters 4
  relations  4
    observed 3  (interaction_passages)
    asserted 1  (asserted_statements)
  rejected   6 of 37 quotations as not verbatim
```



Draft one. Three people and the triangle between them. Mrs Ferrin is in the cast list and in chapter two, but this reading found her sharing no passage of contact with anybody, so she sits alone and pale. Small enough to check by hand — which is the point of starting small.

> **WHAT TO LOOK AT**
>
> Two things. First, the split between *observed* and *asserted*: three relationships were shown happening, one was merely stated. Second — and this is the error promised above — open Auber Vance's panel and look at his aliases. The small local model has folded "Neve Vance" into her father. Remember it; milestones 9 and 10 undo it.

MILESTONE 4 Reading the picture properly

Why. Nothing here costs anything, and it is where most of your time will be spent.

Click the heaviest line — Auber and Idris. The panel gives its weight, its basis, and eight passages of evidence. Click a quotation and the chapter opens beside the graph with the sentence highlighted. Try the layouts. Pin the one you like. Drag Neve somewhere more sensible and watch the position stick.

> **WHAT TO LOOK AT**
>
> The distance between "the graph says these two are close" and "here is the sentence that says so" should be two clicks. If a claim looks wrong, this is how you convict it.

MILESTONE 5 The rewrite

Why. This is the whole reason for the two time axes. You have rewritten chapter three: Auber no longer confronts Idris directly — the confrontation is relayed through Neve. Does the graph show it?

```
# after editing chapter-03.md in your own editor
dramatis ingest . --work "The Lamplighter's Daughter" --label "draft two"
```

```
ingested: rev:3fbe3124d583 (4 documents, 3,869 characters)
  1 changed, 3 unchanged
  against     rev:568a07982816

  unchanged cast.md
  unchanged chapter-01.md
  unchanged chapter-02.md
  changed   chapter-03.md
```

Now re-read it, holding the reading exactly as it was:

```
dramatis analyse rev:3fbe3124d583 --provider ollama --like snap:bb9660a6fe4b --label "draft two"
```

```
note: holding the analysis as recorded by snap:bb9660a6fe4b
      (effort=medium, max_rejection_rate=0.25, target_characters=12000)
```

> **WHAT TO LOOK AT**
> "1 changed, 3 unchanged" — Dramatis knows precisely what you touched. And the note about holding the analysis still is what makes the next milestone mean anything.

MILESTONE 6 The comparison

Why. To find out whether the rewrite did what you intended.

Refresh the browser. The lineage grid now has two rows in one column. Click draft one, then hold **Shift** and click draft two.

> **WHAT TO LOOK AT**
> The first line of the panel, before anything else. Here it reads *"The analysis was held still, so these changes belong to the text."* That sentence is what licenses everything below it, and you get it only because milestone 5 used `--like`. The second line is the tool declining to do something: the two snapshots weigh their relationships on different bases, so nothing is reported as strengthened or weakened rather than a wrong number being reported confidently. Then read the changes and ask whether they match the edit you made.

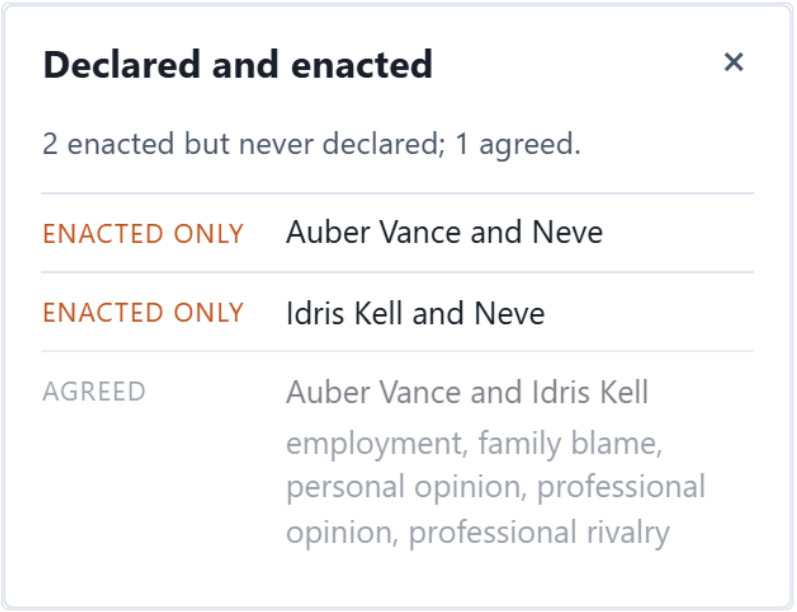
MILESTONE 7 The plan against the page

Why. You have kept a cast list. It makes claims. Do the chapters bear them out?

With a snapshot open, press **Compare declared with enacted** in the sidebar.

WHAT TO LOOK AT

The summary line: here, *"2 enacted but never declared; 1 agreed."* Two of the three relationships the chapters carry are missing from the cast list — an outline that has fallen behind the writing, which is the ordinary condition of an outline. Anything purple and dashed would be the opposite case, a relationship described and never dramatised.



Disagreements first, agreement last and faint. No weights appear anywhere here: passages of contact and statements in a cast list are different quantities, and printing them side by side would invite a comparison that means nothing.

MILESTONE 8 Saying what you make of it

Why. A reading is a proposal. Go through it once and record what you actually think, so that next time you only have to look at what is new.

```
dramatis review
dramatis review --relation rel:auber-vance--neve --status accepted
dramatis review --character char:mrs-ferrin --status rejected \
    --note "a walk-on; not a member of the cast"
dramatis review --pending
```

WHAT TO LOOK AT

The tally: *"7 subjects - 5 proposed, 1 accepted, 1 rejected; 2 ruled on by a person."* None of this touched the snapshot. Your decisions are keyed to the characters and relationships themselves, so the next reading of this work will arrive with them already attached.

MILESTONE 9 Putting the alias mistake right

Why. Milestone 3 found the model folding Neve into her father. There are two separate wrongs to fix: what the graph *says*, and who the next reading thinks these people *are*.

First, what it says:

```
dramatis correct --character char:auber-vance --field aliases \
    --value "Auber" "the lamplighter" \
    --note "Neve Vance is his daughter, not another name for him"
```

Then, who they are:

```
dramatis split char:auber-vance --form "Neve Vance" --name "Neve Vance"
dramatis merge char:neve-vance --into char:neve
```

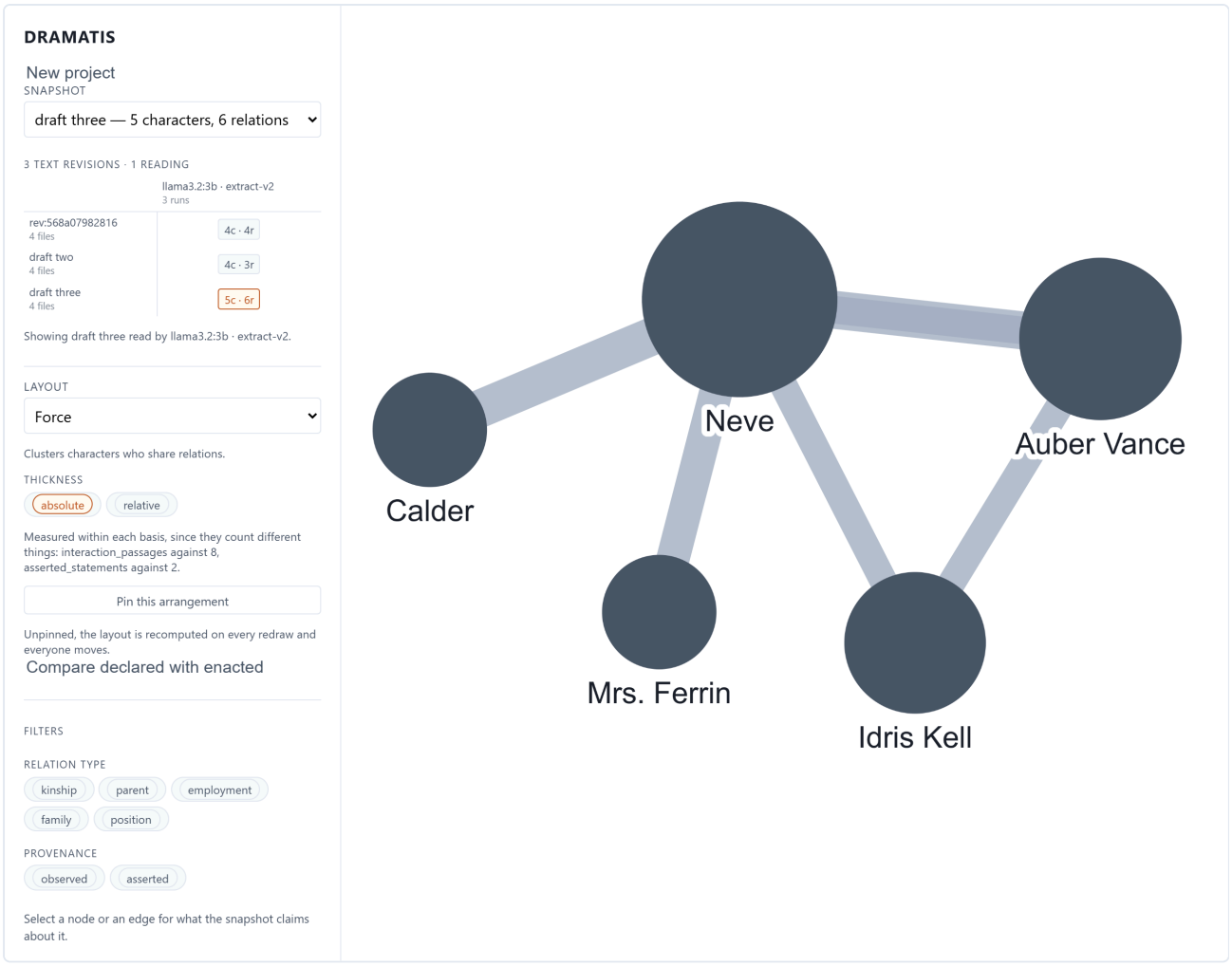
> **WHAT TO LOOK AT**
> The warning Dramatis prints on the split: *"a standing correction governs this character's aliases … so the moved names will reappear there until that correction is updated."* The two tools interact and the tool says so. Then run `dramatis characters`: Neve now answers to "Neve Vance", and the decisions are listed at the bottom with the date and the reason.

MILESTONE 10 Watching the corrections land

Why. A correction that does not survive the next analysis is worth nothing. This is the proof.

```
dramatis ingest . --work "The Lamplighter's Daughter" --label "draft three"
dramatis analyse rev:ba710042c22a --provider ollama --like snap:243d20a2f047 --label "draft
three"
```

```
note  this reading gives character char:auber-vance aliases=["The Lamplighter's Daughter"],
      but a correction holds ["Auber", "the lamplighter"]. The correction stands.
```



Draft three, with every decision in force. Neve now carries the name that used to be filed under her father, and Mrs Ferrin has a relationship the earlier readings missed. “Calder” is the inspector under his new name — and the fact that he is a separate circle from Idris Kell is precisely the problem milestone 11 goes looking for.

> **WHAT TO LOOK AT**
> Open Auber Vance in the new snapshot. *Also known as* now reads “Auber, the lamplighter”; provenance has become `human`; and a block headed *this reading disagreed* records that the model proposed something else and lost. Neve carries the merged name. You made those decisions once and they are still in force.

MILESTONE 11 What the corpus no longer agrees about

Why. Between drafts two and three you renamed the inspector in chapter three and nowhere else. Nothing about the graph looks wrong; the mistake is invisible to a reading.

```
dramatis continuity
```

> **WHAT TO LOOK AT**
> *“‘Kell’ is gone from chapter-03.md, but still appears 5 time(s) elsewhere”*, with every remaining occurrence quoted in context. This is the one finding in Dramatis that no amount of re-analysis would give you, because a cast with a stale name in it looks exactly like a cast. Fix the other files, re-ingest, and run it again until it reports nothing.

MILESTONE 12

Out of Dramatis, into everything else

Why. The study is finished and the argument goes in an article. The figure wants laying out in Gephi, the reviewers want the evidence checkable, and neither of them is going to install this.

```
dramatis export gexf -o lamplighter
dramatis export annotations -o lamplighter
dramatis export snapshot -o lamplighter
```

> **WHAT TO LOOK AT**
>
> Open `lamplighter.gexf` in Gephi: the graph arrives with its weights, and the import panel shows the citation line naming the text revision and the run that read it. Then look at what your curation did to the file — the character you rejected at milestone 9 leaves as `rejected`, not as the `proposed` the model first wrote, because the export is the copy that gets cited. `lamplighter.annotations.jsonld` is the evidence on its own, one annotation per quotation, resolvable by anything that speaks Web Annotation. And `lamplighter.dramatis.json` is the reading itself, which a colleague can `import` into their own project — carrying every claim and every quotation, and not one word of the manuscript.

AND THEN

Keeping it

```
cp dramatis.sqlite ~/backups/lamplighter-2026-08-20.sqlite
```

That is the entire backup procedure. To hand the study to a colleague, send them that file: they can open every snapshot, follow every quotation into the text, read every decision you took and why, and compare every draft — with no key, no network and no account. To hand it to a colleague who should *not* have the manuscript, send them the exported document instead.

What a longer project looks like

Beyond eleven milestones the pattern simply repeats. A realistic sequence for a novel written over a year:

WHEN	WHAT YOU DO	WHAT IT COSTS
At the end of each draft	Ingest, then analyse with `--like` the previous snapshot.	One analysis
After a structural cut	Ingest and analyse; compare against the snapshot before the cut.	One analysis
After every rename	Run `continuity` before you forget you renamed anybody.	Nothing
Whenever a reading is wrong	`correct` what it says; `merge` or `split` who it thinks people are. Once, not once per analysis.	Nothing
Whenever the cast list changes	Re-ingest and re-analyse; check declared against enacted again.	One analysis
Once, out of curiosity	Re-analyse an *old* revision with a better model, to see how much of a past finding was the reading rather than the writing.	One analysis

Continuously	Open, read, filter, follow evidence, review, compare.	Nothing

PART VII

Living with it

Where the files are, how to keep them safe, what it costs, what it promises about privacy, what to do when something breaks, what is genuinely not built yet, and how to remove every trace of it.

33 Where everything lives

WHAT	WHERE	MATTERS?
Your project	`dramatis.sqlite`, in your manuscript folder unless you named it otherwise	**Everything.** Back this up.
Your manuscript	Wherever you keep it — on disk, or in Drive. Dramatis never writes to it.	Yes, but it is yours already
The application	The folder you cloned into, and its `.venv`	No — re-installable in five minutes
Your Google credential	Your own configuration directory, owner-readable only. Deliberately *not* in the project file.	Treat as a password
Checkpoint files	Wherever you pointed `--checkpoint`	Contain your text; delete when the run is done
Pinned layouts	Your browser's local storage	No — cosmetic, and per browser
Your API key	Your shell environment or profile	Treat as a password

Inside the project file

One SQLite database holding your texts (the full content, not a reference to a path on somebody's laptop), the character registry, every snapshot as a complete, valid, self-contained document, and every decision you have taken about them — reviews, corrections, merges and splits, each with its date and its reason. Nothing points outside the file, and no credential is ever in it. That is what makes it portable, archivable and citable.

When one file is the wrong shape

For several people reading one corpus, or a container with no persistent disk, point `--store` at a PostgreSQL address instead. Nothing else changes:

```
pip install -e ".[postgres]"
dramatis ingest draft.txt --store postgresql://user:pass@localhost/dramatis --work "My Novel"
```

A store is chosen once, not migrated between backends. The single file remains the default and the shape the application is designed around.

34 Backing up, sharing, archiving, citing

Backing up

Copy the file. Do it whenever you have run an analysis worth keeping or spent an afternoon reviewing one — those are the two expensive things in the project:

```
cp dramatis.sqlite ~/backups/my-novel-2026-08-20.sqlite
```

Sharing

Send the file. Your recipient needs Dramatis installed but needs no key, no account and no network to read everything in it: the graphs, the evidence, the source text, the comparisons, and every judgement you recorded.

Where the manuscript is the part you cannot send, send a document instead:

```
dramatis export snapshot -o my-reading
```

That is one reading — every character, every relationship, every quotation and every review decision — with a fingerprint for each source document and not a word of the text. Your recipient imports it and works with it; what they cannot do is open the passage around a quotation, because that needs the manuscript you deliberately kept. Part V has the detail.

> THE FILE CONTAINS YOUR MANUSCRIPT
>
> Document contents are stored inside the project so that evidence can always be resolved. Sending the project file sends the text. That is usually what you want and occasionally very much not. It does *not* contain your API key or your Google credential — those live outside it, on purpose.

Archiving and citing

A snapshot is a complete document conforming to a published, separately versioned schema, so it can be deposited and read by other tools without Dramatis. To make a result citable, name three things together:

- the **snapshot** identifier (for example `snap:bb9660a6fe4b`),
- the **text revision** it read,
- the **analysis run** that read it — which carries the model and the version of the instructions.

All three are in the sidebar under any open snapshot. Given them and the project file, another scholar can reproduce exactly what you saw — and can see which of it was the model's proposal and which was your judgement, because those are recorded apart.

For a deposit rather than a handover, export instead of sending the project. `dramatis export snapshot` writes the reading as a document against the published schema; `jsonld` writes it as linked data; `annotations` writes the evidence as W3C Web Annotation, which is the form a reviewer can check without installing anything. All three carry the three identifiers above, so a deposited file says what it is a reading of without needing this paragraph beside it.

35 What it costs and how long it takes

Only `analyse` costs anything, and `structure --ask` if you use it. Everything else — ingesting, browsing, filtering, comparing, reviewing, correcting, merging, continuity, exporting, validating — is free and offline, permanently.

	HOSTED MODEL	LOCAL MODEL
Three chapters	Pennies; under a minute	Free; a few minutes
A full novel	A few dollars at medium effort; several minutes	Free; hours on an ordinary laptop
Every re-read	The same again — the work is not cached between runs	The same again

Four ways to spend less:

- **Exclude what you do not need.** A critical preface bound into the same file, or a production schedule sitting in the folder, has its own cast. Leaving it out costs nothing and saves a real fraction of a long book.
- **Use `--checkpoint` for long runs.** An interrupted novel resumes instead of paying for the same calls again.
- **Start at low effort.** Read a cheap pass, decide whether the shape is right, then pay for a careful one.
- **Correct rather than re-run.** If one character is wrong, correcting it is free and permanent. Re-analysing in the hope the model does better is neither.

36 Privacy, in detail

Precisely what leaves your machine, and when.

ACTION	WHAT LEAVES YOUR MACHINE
`ingest` from disk, `status`, `validate`, `characters`, `review`, `correct`, `merge`, `split`, `correspond`, `continuity`, `export`, `import`, `serve`	**Nothing.** Ever. These work offline and always will.
`structure` without `--ask` or `--drive`	Nothing.

`structure --ask`	The opening of each document, to your chosen model provider.
`analyse` with a hosted model	Your text, in passages, to that provider and nowhere else.
`analyse --provider ollama`	Nothing, provided Ollama is on this machine — and Dramatis warns you if it is not.
`authorise`	Your own OAuth client details, to Google, in exchange for a read-only grant. No part of your corpus.
`ingest --drive`, `structure --drive`	A request to Google for that folder's documents, and the documents come back. One host, every request a read.

Five further guarantees:

- **No telemetry, no analytics, no update checks, no crash reports.** There is no code in Dramatis that contacts Kestora Labs.
- **A path is never inspected to see whether it might be a Drive folder.** Reaching Google requires `--drive`, explicitly, every time.
- **The browser interface binds to this machine only.** Serving it to the network is possible, deliberately awkward, and warned about in plain words.
- **The server refuses writes from other websites.** Any page you have open could send a command to an address on your own machine; every write — including a review, a correction and a merge — is checked and a request from elsewhere is refused before it reaches anything.
- **Each destination has exactly one small module that can reach it**, using nothing but what Python already ships. The property is checkable by anyone who wants to read the source, which is why it is stated this way rather than promised.

> AN EXPORT IS A FILE YOU CAN THEN SEND ANYWHERE
>
> Exporting reaches no network, but it does write your reading to disk, and what happens to that file afterwards is on you. The four graph formats carry names, notes and counts, and not one quotation. The `snapshot` document carries the quotations and a fingerprint per file, but no source text. `annotations` carries every quotation the reading rests on, which is the point of it and also the most of your manuscript any export contains. If a corpus is confidential, that is the export to think about before attaching.

37 When something goes wrong

WHAT YOU SEE	WHAT IT MEANS, AND WHAT TO DO
`dramatis: command not found`	The virtual environment is not active. Run the `activate` line from *Installing, step by step* in this terminal window.

`error: no project file found`	You are outside the project. `cd` into the manuscript folder, or pass `--store` with the path. Dramatis refuses rather than creating an empty project — deliberately.
A page saying the client has not been built	The two `npm` commands from the install were not run, or the source has been updated since. Run them again.
Analysis fails on a credential	`ANTHROPIC_API_KEY` is not set in *this* terminal window. It does not survive closing the window unless you put it in your shell profile.
`Ollama … did not answer within 600s`	A local model on modest hardware can exceed the per-call limit. The message suggests raising the timeout; there is currently no option that does. Use a smaller or faster model, and check that nothing else is competing for the machine.
`too many quotations failed verification`	More than a quarter of what the model produced was not in your text, so the whole run was refused rather than delivered thin and misleading. Try a better model or a higher effort. If it persists, the text may have an unusual encoding.
Two names that are obviously one person	Alias resolution missed a merge. `dramatis merge` settles it permanently; a more capable model avoids most cases in the first place.
One node that is obviously two people	`dramatis split`, naming the forms to move out.
A correction that "did not work"	It did. A correction is written into the *next* snapshot, never into the one you were looking at, because snapshots are immutable. Analyse the work again and it will be there. The panel says so, under the correction.
A character you do not recognise	Usually front or back matter, or a file that is no part of the work. Mark it *excluded* and analyse again.
"This quotation is not in this revision"	Working as designed. You edited the sentence after the analysis. Re-analyse the new revision when you are ready.
The comparison says it cannot attribute the change	Both the text and the reading moved. Re-run the newer analysis with `--like` the older snapshot and compare again.
A Drive ingest reports collisions	Two documents in the folder share a name; only one was read. The message currently names the wrong one of the pair — check the document count against what you expect before analysing.
`dramatis authorise` is refused	The client secret must be for an OAuth client of type **Desktop app**, with the Drive API enabled on that Google Cloud project.
An analysis was interrupted	Run it again with the same `--checkpoint` file and it resumes from where it stopped.
`note: skipped ... : not a text file`	Informational. Dramatis reads `.txt`, `.md`, `.markdown` and `.text`; anything else is named and left out rather than dropped silently.

38 What Dramatis cannot do yet

Named honestly, so you can plan around them, and listed in roughly the order they are expected to arrive. The four the second edition listed first — exports, evidence as standard annotation, importing another tool's graph, and several editions of one work — have all since been built, and Part V is where they are.

NOT YET	WHAT IT WILL BE
A formal citation record	A `CITATION.cff`, a DOI on release, and versioned schema documentation.
A documentation site	This manual, on the web, with the schema reference and prompt documentation beside it.
A double-clickable application	A signed desktop installer for macOS and Windows, with no terminal at all.
Editing the instructions	Per-project prompts, so a scholar can state and publish exactly how their corpus was read, with an evaluation harness to show a change was an improvement.
Merge, split and correspondence in the browser	The server already accepts them; the client has no control for them yet, so they are command-line work for now. Review and correction are in both.
Exporting from the browser	`dramatis export` writes every format there is, and the window has no button for any of them. One endpoint and one control away.
Comparing two editions in the window	The comparison itself is built and the interface behind the window performs it; what is missing is a way to choose two snapshots from two editions, because the screen that picks the pair is a single work's grid.
A figure renamed twice	A correspondence relates a pair of characters. Three editions with two renames need a chain, which is a different structure and is not built.
Smaller things, listed but unscheduled	A `--timeout` for slow local models; reading HTML as text; pruning a subtree from a Drive walk; a `diff` command outside the browser; noticing when a local model read less than it was sent.

39 Uninstalling, completely

Nothing is hidden anywhere on your system. Removing Dramatis is removing one folder, and you decide separately what to do with your projects.

1 · Decide about your projects first

Your `dramatis.sqlite` files are in your own manuscript folders and are not touched by any of the steps below. Each holds analyses you paid for and judgements you made. Copy them somewhere safe if there is any chance you will want them, or delete them deliberately.

```
# find every project on the machine (macOS and Linux)
find ~ -name "dramatis.sqlite" 2>/dev/null
```

```
# Windows PowerShell
Get-ChildItem -Path $HOME -Filter dramatis.sqlite -Recurse -ErrorAction SilentlyContinue
```

2 · Give back the Drive grant, if you gave one

```
dramatis authorise --forget
```

That deletes the cached credential from this machine. It does not revoke the grant at Google — do that under *Third-party apps with account access* in your Google account settings, and delete the OAuth client in the Cloud console if you made it only for this.

3 · Remove the application

Delete the folder you cloned into. That is the virtual environment and all of it:

```
rm -rf ~/dramatis          # macOS and Linux
```

```
Remove-Item -Recurse -Force $HOME\dramatis      # Windows PowerShell
```

4 · Remove what you added for it

- **Your API key.** Delete the `ANTHROPIC_API_KEY` line from your shell profile, and revoke the key itself in the provider's console — deleting the line does not disable the key.
- **Checkpoint files.** Any file you passed to `--checkpoint`. These contain your text.
- **Ollama**, if you installed it only for this. Uninstall it like any other application; `ollama rm llama3.1` first reclaims the model's disk space.
- **Python and Node**, if you installed them only for this and nothing else on your machine uses them. Most people should leave them.
- **Pinned layouts** live in your browser's storage for `127.0.0.1:7373` and disappear when you clear site data. They are cosmetic.
- **The Docker image**, if you used it: `docker rmi dramatis`.

> THERE IS NOTHING ELSE
>
> No system-wide configuration, no registry entries, no background service, no login item, no account to close, and no remote copy of anything — because nothing was ever sent anywhere except to the model provider you chose and, if you asked for it, the Drive folder you named.

END MATTER

Glossary and credits

A Glossary

Alias	A surface form of a name: “Lizzy” for Elizabeth Bennet. Gathering aliases onto one character is *alias resolution*, and it is the hardest thing the pipeline does.
Analysis run	One reading: a model, a version of the instructions, and the settings used.
Annotation	One piece of evidence in W3C Web Annotation form: a quotation, the text either side of it, where in the document it sits, and which claim it supports. What `export annotations` writes, one per quotation.
Asserted	A claim your reference material states, as opposed to one the narrative enacts.
Attribution	A comparison's verdict on *what* a difference can be blamed on: the work, the reading, the edition, or neither.
Checkpoint	A file recording every model call, so an interrupted analysis resumes rather than paying twice. Contains your text.
Collection	The circle inside which characters are shared. One per project.
Confidence	How sure a reading was, between 0 and 1. Optional, undeclared in what it counts, and absent from every reading Dramatis currently produces. Below 0.50 is drawn dotted; an absent value is not a low one.
Conflict	A later reading disagreeing with a correction you made. The correction stands and the disagreement is recorded and shown.
Continuity report	What the corpus no longer agrees with itself about: stale names, citations with nowhere to land, a replaced draft still being read.
Correction	A statement of what a claim should say, applied to every snapshot built afterwards. Makes the claim `human`.
Correspondence	A declaration that a character in one edition and a character in another are one figure. Changes neither of them, which is what distinguishes it from a merge.
Document	One file in a corpus, with a role: narrative, reference, or excluded.
Edition	A published state of a work that supersedes nothing. Part of the work's identity, so two editions are two works sharing one collection and one cast, both current and both citable.

Evidence	A quotation with a locator and often a note, attached to a relationship. Verified verbatim against the source or discarded.
Excluded	A document that is in the folder and is no part of the work. Left out of the revision entirely rather than stored as reference material nobody wanted read.
Export	A reading written out for something else to read: GraphML, GEXF, CSV, JSON-LD, the evidence as annotation, or the stored document itself. Standing review decisions are applied on the way out of all but the last.
Import	Reading a Dramatis document into a project. Refuses before it writes if anything in it collides with what is already there.
Ingest	Reading text into a project as a fingerprinted revision. Never calls a model.
Lineage	A work's snapshots arranged on the two time axes.
Locator	Where in a document something is: an ordered path of typed segments. The types are data supplied per work, so the schema names no chapter, panel or scene.
Merge	Declaring two registered characters to be one person. Takes effect through the registry, on the next reading; no stored snapshot changes.
Observed	A claim the narrative enacts on the page.
Provenance	Where a claim came from: observed, asserted, or human.
Re-anchoring	Finding a quotation again after the text around it has been edited — exactly, then by surrounding context, then approximately, and never silently.
Region	A marked part of a document, which may be excluded so it is never stored or analysed. What a preface goes in.
Registry	The collection's cast: every character and every surface form it answers to. What merge and split edit, and what the next reading resolves names against.
Review status	How far a claim has travelled through human checking: proposed, accepted, corrected, rejected. Appended, never overwritten.
Role	Whether a document enacts the story, describes it, or is no part of it.
Schema	The published, separately versioned definition of a Dramatis document, so other tools can read and write the format without running Dramatis.
Segment	A division of a text. Dramatis currently divides on blank lines and calls each block a *section*.
Snapshot	A graph bound to one text revision and one analysis run. Immutable.
Source	Where a corpus comes from: a folder on disk, or a Google Drive folder. Named without being contacted; contacted only when read.
Split	Declaring one registered character to be two people, by moving some of its names to a new one. The only way to undo a merge.
Structure map	Dramatis's proposals about a corpus, with evidence, awaiting your confirmation.
Text revision	The exact fingerprinted state of a work's files at one moment.

Weight basis	What a weight counts. Weights are comparable only within a shared basis, and Dramatis refuses to compare across bases rather than producing a wrong answer.

B Licences and credits

Dramatis is a Kestora Labs product. The code is Apache-2.0; the documentation, including this manual, is CC BY 4.0. The source is at github.com/kestora-labs/dramatis, and it publishes as `dramatis-personae`.

Dramatis ships no model and no credentials. Whichever model you use is yours, under that provider's terms, and the Google client you create for Drive access is yours too.

The screenshots of *Pride and Prejudice* are from the Project Gutenberg text, which is in the public domain. *The Lamplighter's Daughter* is synthetic, written for this repository's test fixtures, and deliberately short.

Dramatis — The Manual, third edition. Describes version 0.1.0.dev0, built through Phase 6.4. Everything stated here was checked against the running application; every screenshot is of a real analysis of a real text, and every quoted output is a real transcript. The Salt Road transcripts in Part IV are of fixture **D**, which is synthetic and written for this repository.

Further reading in the repository: `docs/schema.md` for the document format, `AI/ROADMAP.md` for the design specification and the invariants every change must respect, `DECISIONS.md` for the reasoning behind the choices that shaped it, and `AI/FINDINGS.md` for what using it on real corpora has turned up.